

Streaming Concept-Drift Detection on a Multimodal Classification Stream: Technical Report

Dominik Zieliński

June 8, 2026

Abstract

This report presents an end-to-end streaming benchmark for detecting controlled distribution changes in a multimodal classification stream. Each stream instance consists of a real image, a generated short caption and a class label. The stream generator controls the temporal class/source composition, the embedder maps every image–text pair to a fixed-dimensional feature vector, and the monitoring layer detects drift either directly in the embedding distribution or indirectly through the prequential error of an online classifier. The report therefore separates 3 linked questions: (1) how the stream is constructed, (2) which signal is monitored for drift, and (3) whether a drift alarm improves downstream adaptation.

The benchmark includes 4 within-dataset drift schedules, 3 data-drift detectors based on the Fréchet distance [4], Maximum Mean Discrepancy [10] and projected Kolmogorov–Smirnov windowing [19], and 4 performance-drift detectors driven by online classifier errors: KSWIN, ADWIN [1], Page–Hinkley [18] and a weighted Hoeffding-bound detector [5]. Logistic regression, Hoeffding Tree and Gaussian Naive Bayes are evaluated as streaming classifiers. The final experiments show that embedding-level monitoring is generally more reliable than error-only monitoring for soft within-dataset class-prior drift, while a stronger source-expansion drift makes both the embedding distribution and the classifier error stream change more visibly. However, the results also show that detecting drift does not by itself justify replacing the active classifier: if earlier classes remain in the post-drift stream, a full reset can discard useful accumulated knowledge.

1 System overview

The pipeline processes a multimodal stream sample-by-sample. For every time step t the stream emits 1 image, 1 text caption and the corresponding class label. An embedder maps the pair to a vector $x_t \in \mathbb{R}^D$. An optional online classifier consumes (x_t, y_t) in test-then-train (*prequential*) mode [7]. The configured drift detector is updated. Each detector update returns a record that contains, when defined, the comparison window indices, a scalar drift score, the current decision threshold, and binary drift and warning flags. The 2 monitoring regimes share this interface: in the data regime the detector input is the embedding itself, whereas in the performance regime the input is the per-sample classification error of an online classifier.

Cause–result structure of the report. The experimental logic is as follows. (1) The stream generator changes the class or source composition over time. This produces a measurable change in the distribution of image–text embeddings and, depending on the classifier and dataset, may also perturb the online prediction-error process. (2) Data-drift detectors act on the embedding-distribution consequence directly, whereas performance-drift detectors observe only the compressed prediction-error consequence. The classifier replacement analysis is therefore deliberately kept separate from detector quality: a good alarm signal can still lead to worse macro- F_1 if the adaptation rule discards knowledge about classes that remain relevant after drift.

Terminology used in diagnostic plots. Throughout this report, *predicted detection* denotes a time step at which the detector raised a true drift alarm (`is_drift` = 1). In the diagnostic stream plots, these alarms are drawn as magenta vertical dashed lines. The quantitative metrics are computed from the complete alarm stream: no post-hoc cap is placed on the number of alarms that a detector may raise.

2 Multimodal stream construction

The stream is *semi-synthetic*: real images are drawn from a folder-based dataset (1 subdirectory per class), and a short pseudo-random caption is generated for every sample so that the text modality is also non-trivial.

Each sample stores the time index, image path, label, label index, generated text and phase tag. The phase tag records the drift regime the sample belongs to and is used only for visualisation, not by the detectors themselves. Class identifiers (the integer *label index*) are assigned once, statically, in the sorted order of the class subdirectory names: for PetImages this yields Cat=0, Dog=1. This static mapping is independent of the order in which samples appear in the stream. The stream itself is built sample-by-sample: at each step the schedule selects the dominant class at that time before the actual label is drawn from a categorical distribution over all classes (Equation (1)), and the corresponding image is then drawn from that class’s pool. As a result, individual samples of different classes are interleaved within every drift segment. All images of 1 class are *not* grouped together. The pseudocode below summarises this construction.

Algorithm 1: Sample-by-sample construction of the semi-synthetic multimodal drift stream.

Input: class folders $\{C_1, \dots, C_C\}$, total length N , drift schedule $\mathcal{S}$, per-step dominant class $\ell^*(t)$ and dominant probability p^* , text generator $\text{TextGen}(\cdot)$, RNG seed.

Output: stream $\{(t, \text{img}_t, y_t, \text{id}(y_t), \text{txt}_t, \text{phase}_t)\}_{t=0}^{N-1}$.

```

1 Build class-ID map  $\text{id}(\cdot)$  from sorted class names.
2 Validate each candidate image (open + parse) and form per-class pools  $\mathcal{P}_c = \text{shuffle}(C_c)$ 
  with per-class cursor  $\kappa_c \leftarrow 0$ .
3 for  $t = 0, \dots, N - 1$  do
4    $\ell \leftarrow \ell^*(t)$  (dominant class at step  $t$  from  $\mathcal{S}$ )
5   Form distribution  $w(t)$  via Equation (1) (or the interpolated form Equation (2) inside
     a gradual ramp)
6   Draw label  $y_t \sim \text{Categorical}(\{C_1, \dots, C_C\}, w(t))$ 
7    $\text{img}_t \leftarrow \mathcal{P}_{y_t}[\kappa_{y_t}]$     $\kappa_{y_t} \leftarrow \kappa_{y_t} + 1$            // no replacement within class
8    $\text{txt}_t \leftarrow \text{TextGen}(y_t)$ 
9   Record phase tag from  $\mathcal{S}$  (e.g. pre_drift, gradual_drift_i, ...)
10  Emit  $(t, \text{img}_t, y_t, \text{id}(y_t), \text{txt}_t, \text{phase}_t)$ 
11 end
```

2.1 Image stream

Image discovery is followed by a validation pass that opens each candidate image and verifies that it can be parsed, so that corrupt files are filtered out before they enter a class pool. Pools are shuffled with a deterministic random generator and each class draws images without replacement using an incrementing per-class cursor. Therefore, provided that every class pool contains enough validated images for the requested schedule, the stream never reuses the same image. Once the drift schedule has fixed which class label provides the image at each step, the corresponding path is taken from that class pool and stored in the manifest. The actual embedding is computed lazily by the streaming pipeline at inference time, not at stream construction time.

2.2 Text stream

The textual companion of each sample is generated from a fixed vocabulary of short generic tokens (descriptive everyday words and a few stream-related terms). For each sample, the caption length is sampled uniformly from a configurable range and tokens are drawn independently. A separate label-specific vocabulary contains a small set of *manually curated class-aware tokens*. These tokens are merged into a caption with a controllable probability p_{txt} : each token position is independently replaced by a class-specific token with probability p_{txt} . The class-aware tokens are not obtained by image captioning or by extracting objects from individual images. They are hand-specified word lists keyed by class name, for example $\{\text{cat}, \text{kitten}, \text{whiskers}, \text{paws}, \text{meow}, \text{feline}, \text{tabby}\}$ for the **cat** class and $\{\text{dog}, \text{puppy}, \text{bark}, \text{canine}, \text{leash}, \text{retriever}, \text{terrier}\}$ for the **dog** class. If a label has no curated entry, the generator falls back to 1 token derived from the class name. Setting $p_{\text{txt}} = 0$ makes the caption channel statistically independent of the label. Setting $p_{\text{txt}} > 0$ injects a controlled label-correlated signal into the text modality.

2.3 Multimodal embedding

The embedder supports 3 modalities:

- **Image only.** A pre-trained ResNet-18 [11] convolutional network is used as a fixed feature extractor: the classification head is removed and the global-average-pooled feature map is taken as the embedding (dimension 512). Inputs are pre-processed with the standard ImageNet-style protocol [3] (resize to 224×224 , channel-wise normalisation). All experiments reported in Section 9 use this ResNet-18 feature path.
- **Text only.** A feature-hashing vectoriser [21] maps each caption to a sparse fixed-dimensional vector without maintaining a vocabulary, with no sign cancellation and without normalisation. The result is materialised as a dense vector per sample.
- **Multimodal.** The image and text vectors are computed independently, each is ℓ_2 -normalised, multiplied by a per-modality weight, concatenated, and the resulting vector is ℓ_2 -normalised again. With a deep image encoder the final embedding has dimension $D = 512 + 256 = 768$ in the default configuration.

The embedder is constructed once per run and called sample-by-sample inside the streaming loop, mirroring an online deployment: there is no batching across time steps.

3 Within-dataset drift schedules

Drift is injected at the *label distribution* level: at any given time t , the stream sampler picks a class label with a time-dependent probability, and the corresponding image is then drawn from that class’s pool. 4 drift schedules are supported. They share a common low-level mechanism, the *dominant-class* sampler:

$$P(\text{label} = \ell \mid t) = \begin{cases} p^*, & \text{if } \ell \text{ is the dominant class at time } t, \\ \frac{1 - p^*}{C - 1}, & \text{otherwise,} \end{cases} \quad (1)$$

where $p^* \in (1/C, 1]$ is the dominant-class probability and C is the number of classes. Setting $p^* = 1$ recovers pure-class segments

(an abrupt switch from $P(y=\text{Cat} \mid t < k)=1$ to $P(y=\text{Dog} \mid t \geq k)=1$ in the binary case). Values strictly below 1 produce a noisy dominant-class regime where the minority class still

appears at every step with probability $1 - p^*$, which makes both the classification task and the performance-drift signal more realistic. All illustrative drift figures use the deliberately soft setting $p^* = 0.65$ (the default `DEFAULT_IMAGE_LABEL_AWARE_PROB` of the stream builder), so that even an “abrupt” binary label flip changes $P(y=\text{Cat} \mid t)$ from 0.65 to 0.35 rather than from 1 to 0. The final benchmark in Section 9 then sweeps $p^* \in \{0.55, 0.65, 0.75, 0.85\}$ to measure how detector behaviour changes with drift magnitude.

Thus, the within-dataset experiments implement controlled class-prior or mixture shifts rather than geometric transformations of the images conditional on a fixed label. This distinction is important for interpreting the results: data-drift detectors can react because the marginal embedding distribution changes, while performance-drift detectors react only if that mixture shift also changes the online classifier’s error rate.

Conventions in the schedule figures. Figures 1–4 are short diagnostic examples, not the aggregate benchmark. Blue and orange points show the sampled class labels at each time step, the grey row shows the generated text-signal trace, red dashed lines and pink shaded regions mark the ground-truth drift times or intervals, and magenta dashed lines mark predicted detector alarms. The figures are intended to show how the same probabilistic sampler produces interleaved class instances under different drift schedules.

3.1 Abrupt drift

A change point k splits the stream into 2 contiguous blocks: the pre-drift block is dominated by 1 class and the post-drift block by another [8]. Within each block, labels are sampled according to Equation (1) and shuffled with a per-run seed so that the post-drift block does not start with a deterministic alternation pattern.

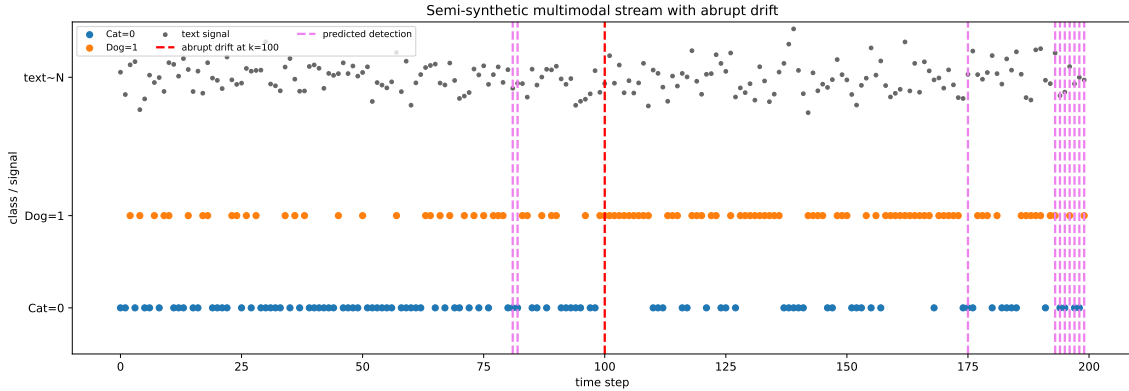


Figure 1: Abrupt drift diagnostic. The red dashed line is the true change point. Magenta dashed lines are detector alarms. Because $p^* = 0.65$, both classes remain interleaved before and after the change, but the dominant class flips at $k = 100$.

3.2 Gradual drift

A linear interpolation between 2 label distributions runs over a closed interval $[k_{\text{start}}, k_{\text{end}}]$, with $k_{\text{end}} > k_{\text{start}}$ [8]. Letting w_{pre} and w_{post} be the dominant-class probability vectors of the 2 regimes,

$$\alpha(t) = \frac{t - k_{\text{start}}}{k_{\text{end}} - k_{\text{start}}}, \quad w(t) = (1 - \alpha(t)) w_{\text{pre}} + \alpha(t) w_{\text{post}}. \quad (2)$$

Inside the gradual interval, $\alpha(t)$ is clamped to $[0, 1]$ and the label at time t is sampled from the categorical distribution $w(t)$. Outside the interval the dominant class is fixed at the corresponding endpoint distribution.

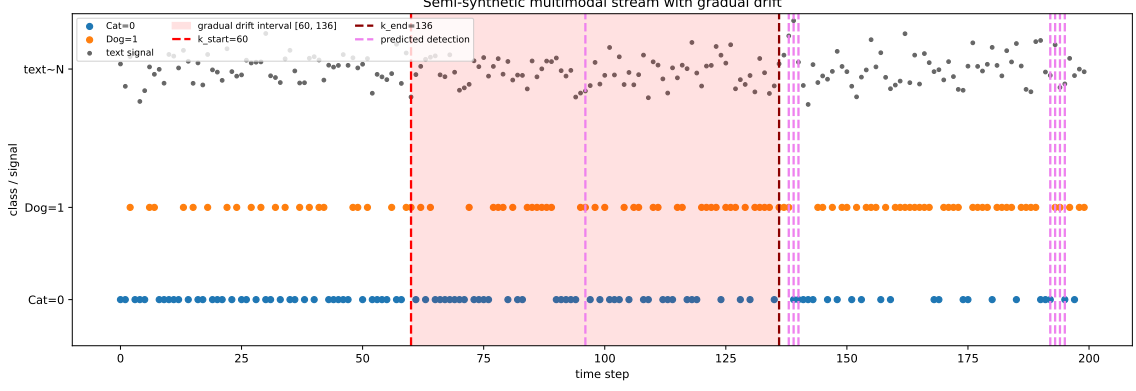


Figure 2: Gradual drift diagnostic. The shaded interval is the true transition window, with red dashed boundaries at k_{start} and k_{end} . Magenta dashed lines are detector alarms. The class mixture changes continuously rather than at a boundary.

3.3 Recurrent (abrupt) drift

A strictly increasing list of change points $K = \{k_1, k_2, \dots, k_M\}$ partitions the stream into $M + 1$ phases delimited by $\{0, k_1, \dots, k_M, N_{\text{total}}\}$ [16]. The dominant class of phase j cycles through the class list as $\ell_j = \text{class}_{(j \bmod C)}$, and each phase uses Equation (1) as its sampler. With $C = 2$ the schedule alternates between the 2 classes. With $C > 2$ it cycles through the class list in order.

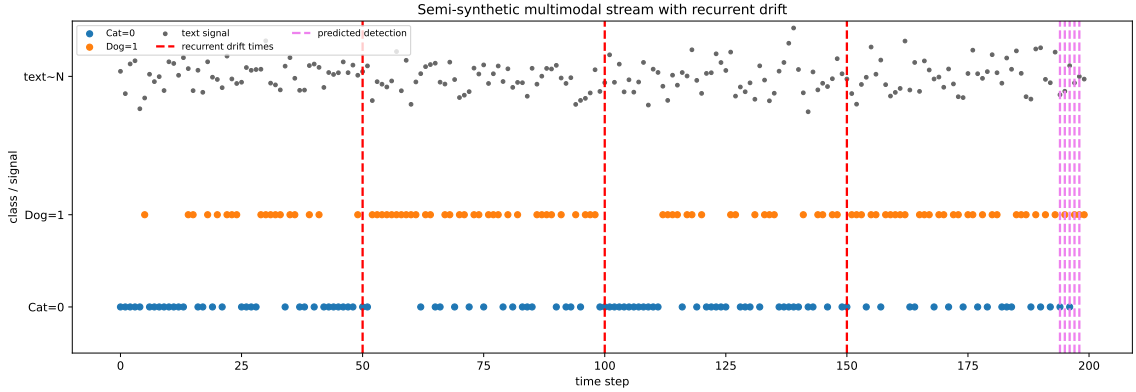


Figure 3: Recurrent abrupt drift diagnostic. Red dashed lines mark successive true change points and the dominant class alternates after each boundary. Magenta dashed lines show the detector alarms raised on this illustrative run.

3.4 Recurrent gradual drift

The schedule is given as a list of non-overlapping ramp intervals $\{(k_{\text{start}}^{(i)}, k_{\text{end}}^{(i)})\}_{i=1}^M$, sorted by start time. The construction proceeds sequentially: a stable block precedes each ramp and is dominated by the current class. Inside the i -th ramp the dominant distribution is linearly interpolated towards the next class in cyclic order, exactly as in Equation (2) but with the endpoints replaced by the current and next class distributions. Once a ramp is finished, the current class is updated to the next class. After the last ramp, a final stable block uses the resulting current class up to the end of the stream. This produces a sequence of slow class flips separated by stable plateaus, a regime that is qualitatively harder than abrupt recurrence because each concept change is spread across many samples rather than marked by a sharp boundary [16].

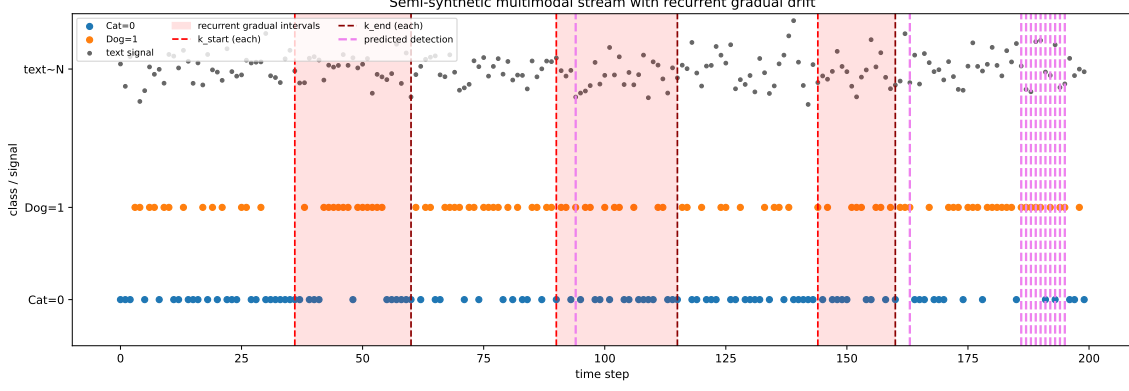


Figure 4: Recurrent gradual drift diagnostic. Each shaded region is a true gradual transition interval, the red dashed lines mark ramp starts and ends, and magenta dashed lines show the detector alarms. This schedule combines recurrent concepts with delayed, ramp-like transitions.

4 Streaming loop and drift-detection metrics

4.1 Streaming pipeline

For every sample, the pipeline (i) looks up the manifest row for time t , (ii) computes the embedding x_t , (iii) if a performance monitor is attached, runs the never-reset online classifier in test-then-train mode and feeds its binary error to the detector, (iv) updates the detector and reads back the drift and warning flags, and (v) updates the replace-on-drift classifier using the *previous* step’s flags so that the strategy stays causal: the detector decision computed from y_t is not allowed to affect the prediction at t itself [7].

The pseudocode below makes this causal order explicit. The detector input is denoted by u_t : in the data-drift regime $u_t = x_t$, while in the performance-drift regime $u_t = e_t^{(1)} = \mathbb{1}[\hat{y}_t^{(1)} \neq y_t]$, the prequential error of the Never-reset model. The Replace-on-drift model can react only to alarms already available from the previous time step.

Algorithm 2: Causal streaming loop used for data-drift and performance-drift monitoring.

Input: stream manifest, embedder Φ , detector D , optional Never-reset classifier $f^{(1)}$, optional Replace-on-drift classifier $f^{(2)}$.

Output: per-step detector scores, alarms, warnings and classifier metrics.

- 1 Initialise detector state. Initialise classifiers when the performance regime is used and set previous warning/drift flags to false.
 - 2 **for** $t = 0, \dots, N - 1$ **do**
 - 3 Read $(\text{img}_t, \text{txt}_t, y_t)$ from the stream manifest.
 - 4 Compute $x_t \leftarrow \Phi(\text{img}_t, \text{txt}_t)$.
 - 5 If the performance regime is active, predict with $f^{(1)}$ before training and set $u_t \leftarrow \mathbb{1}[f^{(1)}(x_t) \neq y_t]$. Otherwise set $u_t \leftarrow x_t$.
 - 6 Update the detector with u_t and record its score, warning flag and drift flag.
 - 7 Train the Never-reset classifier on (x_t, y_t) , if present.
 - 8 Update or promote the Replace-on-drift classifier using the previous step’s warning/drift flags, then train it on (x_t, y_t) .
 - 9 Store the current detector flags for use at time $t + 1$.
 - 10 **end**
-

4.2 Evaluation metrics for drift detection

Because the data stream is synthesised, the true drift onsets are known and the alarms produced by any detector can be scored against this ground truth. The evaluation protocol used in this work follows the standard streaming change-detection setting [8], in which the trade-off between detection sensitivity, detection latency and false-alarm rate is quantified by a small set of complementary metrics.

The metrics below intentionally separate event coverage from alarm volume. Recall asks whether each true event was detected at least once, precision asks how many raised alarms fall inside legitimate event windows, and the latency/false-alarm metrics then penalise late or repeated alarms. This is important for detectors such as Fréchet or MMD, which may fire many times inside 1 long gradual interval.

Notation. Let $K = \{k_1, k_2, \dots, k_M\}$ denote the set of true drift onset times and $\hat{K} = \{\hat{k}_1, \hat{k}_2, \dots, \hat{k}_R\}$ the set of times at which the detector raised an alarm, where each $\hat{k}_j$ is taken to be the right edge of the comparison window that triggered the alarm. For abrupt and recurrent abrupt drift, the onset is the 1 change point itself. For gradual and recurrent gradual drift, the onset is the start of the corresponding ramp. Let H denote the *detection horizon*: an alarm raised more than H steps after the true onset is no longer counted as a successful on-time detection of that event. Let T denote the index of the last sample in the stream.

Recall, precision and F_1 (horizon-based matching). A true drift event i is considered *detected within horizon* if there exists at least 1 alarm $\hat{k}$ that falls inside its associated match interval. For abrupt and recurrent abrupt drift, the match interval of event i is the horizon-bounded interval up to either $k_i + H$ or the index just before the next true onset, whichever is earlier. For gradual and recurrent gradual drift, the match interval is the ramp interval itself, since the onset is intentionally extended in time and any alarm raised inside the ramp is on-time by construction. Letting N_h denote the number of true events that are detected within their match interval and N_a the number of alarms that fall inside any such interval, the metrics are

$$\text{Recall} = \frac{N_h}{M}, \quad \text{Precision} = \frac{N_a}{R}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (3)$$

In words: recall measures the fraction of true drift events that the detector caught at least 1 alarm before the horizon expired. Precision measures the fraction of raised alarms that landed in a legitimate event window. F_1 is their harmonic mean. Repeated alarms inside the same gradual ramp increase precision only if they remain inside the ramp interval, but they do not increase recall beyond 1 detected event.

Detection delay and mean detection delay. For delay-based metrics, the *event-interval rule* is used. The valid interval of event i runs from its onset k_i to the index immediately before the next true onset k_{i+1} (or to T for the last event). The earliest alarm that falls inside this interval is declared the valid detection $\hat{k}_i^{\text{valid}}$ of event i , and all later alarms inside the same interval are flagged as repeated false alarms (see below). The detection delay of event i is

$$\Delta_i = \hat{k}_i^{\text{valid}} - k_i, \quad i \in \mathcal{V}, \quad (4)$$

where $\mathcal{V}$ is the set of validly detected events. The mean and median detection delays summarise the distribution of Δ_i across detected events:

$$\text{MDD} = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \Delta_i, \quad \text{MedDD} = \text{median}(\{\Delta_i\}_{i \in \mathcal{V}}). \quad (5)$$

In words, the mean detection delay is the average number of stream steps a detector takes to react after a true drift onset, computed over the events that were actually detected. By construction $\Delta_i \geq 0$, since the detection cannot precede the onset under the event-interval rule. Smaller values mean a faster reaction to drift.

Missed detection rate. The missed detection rate is the complement of the detection rate under the event-interval rule:

$$\text{MDR} = \frac{|K| - |\mathcal{V}|}{|K|} = 1 - \frac{|\mathcal{V}|}{M}. \quad (6)$$

In words, MDR is the fraction of true drift events for which the detector failed to raise even 1 alarm before the next true onset (or before the end of the stream, for the last event). A perfect detector has $\text{MDR} = 0$. A detector that never fires has $\text{MDR} = 1$. MDR differs from $1 - \text{Recall}$ because it does not require the alarm to fall inside the shorter detection horizon H . It only requires an alarm inside the wider event interval.

Mean time between false alarms. Within the event-interval rule, only the earliest alarm inside a valid interval is credited to its event. Alarms before the initial true onset, alarms outside all match intervals, and subsequent alarms inside an already detected event interval are treated as false alarms. Let $f_1 < f_2 < \dots < f_Q$ denote their times, sorted across the stream. The mean time between false alarms is then the average inter-arrival time of false alarms,

$$\text{MTFA} = \frac{1}{Q-1} \sum_{j=2}^Q (f_j - f_{j-1}), \quad Q \geq 2, \quad (7)$$

and is undefined for $Q < 2$. In words, MTFA estimates how many stream steps elapse, on average, between 2 consecutive false alarms. Larger values mean that the detector raises spurious alarms less frequently.

Mean time ratio. The mean time ratio aggregates the previous 3 quantities into a scalar goodness measure that rewards a detector simultaneously for high detection rate, low delay and a long quiet period between false alarms:

$$\text{MTR} = \frac{\text{MTFA}}{\text{MDD}} \cdot (1 - \text{MDR}). \quad (8)$$

In words, the ratio MTFA/MDD measures how many detection delays fit inside 1 mean inter-false-alarm interval, that is, how *quiet* the detector is relative to how *fast* it reacts. The factor $(1 - \text{MDR})$ scales this ratio down by the fraction of true drifts the detector actually catches. Higher MTR is better. A fast detector that also has rare false alarms and few misses scores high. Slow reaction, frequent false alarms, or many missed events drag the score down. MTR is undefined when MDD or MTFA cannot be evaluated, for example when there is no validly detected event or fewer than 2 false alarms.

5 Data-drift detectors

The detectors that watch the embedding distribution directly are Maximum Mean Discrepancy with a Gaussian kernel [10], the Fréchet distance between empirical Gaussian fits [4], and the Kolmogorov–Smirnov windowing test on a scalar projection of the embedding [19]. MMD and Fréchet share a common 2-window framework with an adaptive, reference-free threshold. KSWIN delegates the windowing to the underlying KSWIN test on a univariate projection.

5.1 2-window framework with adaptive thresholds

For every new embedding, a sliding buffer of length $2W$ is updated and split into a *previous* window $X = \{x_{t-2W+1}, \dots, x_{t-W}\}$ and a *current* window $Y = \{x_{t-W+1}, \dots, x_t\}$. A scalar distance $s_t = d(X, Y)$ is computed and appended to the history of all observed scores. The decision threshold is taken as the empirical *quantile* of past scores excluding the current score. The threshold is purely *adaptive* and *reference-free*: there is no fixed nominal block that defines it, the threshold is a running quantile of the same score distribution being monitored. A drift alarm is raised when the current score exceeds the upper quantile (typically the 0.95 quantile), and a softer warning is raised when the score exceeds a 2nd, lower quantile (typically the 0.85 quantile). A configurable warm-up requires a minimum number of past scores before any decision is emitted, so that the quantile estimate is not trusted before it has stabilised.

This adaptive-quantile rule is a lightweight non-parametric control-chart heuristic: the statistic is a scale-aware distance between 2 adjacent windows, while the decision boundary is defined by the empirical distribution of past statistics rather than by a parametric null distribution.

5.2 Fréchet distance between empirical Gaussian fits

The Fréchet distance is plugged into the streaming pipeline by instantiating the 2-window framework of Section 5.1: the embedder produces 1 embedding vector x_t per stream step, the sliding buffer of length $2W$ is kept up to date, and at every step the 2 adjacent halves of that buffer (the *previous* window $X = \{x_{t-2W+1}, \dots, x_{t-W}\}$ and the *current* window $Y = \{x_{t-W+1}, \dots, x_t\}$) are fitted with their empirical mean and covariance and the squared Fréchet distance between the 2 empirical Gaussian fits is computed as the per-step drift score. The decision is then emitted by the same adaptive-quantile rule used for the other 2-window detectors. With this design, every stream step produces exactly 1 drift score, so the Fréchet distance is well-defined on a stream even though the textbook definition is between 2 probability distributions: each window plays the role of an empirical distribution and the score is the Fréchet distance between the 2 empirical Gaussian approximations.

The Gaussian Fréchet distance [4] between 2 Gaussian distributions with means μ_1, μ_2 and covariance matrices Σ_1, Σ_2 is the squared 2-Wasserstein/Bures distance

$$d^2(\mathcal{N}_1, \mathcal{N}_2) = \|\mu_1 - \mu_2\|_2^2 + \text{tr}\left(\Sigma_1 + \Sigma_2 - 2(\Sigma_1^{1/2}\Sigma_2\Sigma_1^{1/2})^{1/2}\right). \quad (9)$$

For positive semi-definite covariances, the trace term is equivalently the nuclear-norm cross term used in Fréchet Inception Distance [12]. A direct evaluation of Equation (9) requires matrix square roots of $D \times D$ matrices, whose stable numerical computation is typically cubic in D and commonly based on Schur-type methods [13]. For the embedding sizes used here ($D \in \{256, 512, 768\}$) and the small sliding-window sizes typical of streaming ($n = m = W$, with W around 50), both empirical covariances are heavily rank-deficient because $\text{rank}(\Sigma) \leq \min(n, D) - 1$. A sample-space computation is therefore both faster and more stable.

The implementation evaluates the same quantity entirely in the *sample space*. Let $\bar{X}, \bar{Y}$ be the empirical means and define the centred, square-root-normalised data matrices

$$U = \frac{(X - \bar{X})^\top}{\sqrt{n-1}} \in \mathbb{R}^{D \times n}, \quad V = \frac{(Y - \bar{Y})^\top}{\sqrt{m-1}} \in \mathbb{R}^{D \times m},$$

so that $\Sigma_1 = UU^\top$ and $\Sigma_2 = VV^\top$. The covariance cross term can then be computed from the singular values of the small cross matrix $U^\top V \in \mathbb{R}^{n \times m}$:

$$\text{tr}\left((\Sigma_1^{1/2}\Sigma_2\Sigma_1^{1/2})^{1/2}\right) = \|U^\top V\|_* = \sum_i \sigma_i(U^\top V), \quad (10)$$

while the individual covariance traces simplify to the squared Frobenius norms $\|U\|_F^2$ and $\|V\|_F^2$. The reported score is $\hat{d}^2 = \|\bar{X} - \bar{Y}\|_2^2 + \|U\|_F^2 + \|V\|_F^2 - 2 \sum_i \sigma_i(U^\top V)$, clamped to be non-negative to absorb floating-point round-off. The dominant cost is the singular-value decomposition of an $n \times m$ matrix, namely $\mathcal{O}(\min(n, m)^2 \max(n, m))$ and $\mathcal{O}(W^3)$ when $n = m = W$, rather than $\mathcal{O}(D^3)$. This is several orders of magnitude cheaper for the dimensions used here and is also amenable to GPU acceleration.

The score responds to changes in either the mean of the embedding distribution or its 2nd-order structure (the singular spectrum of the covariance), which makes it a useful scalar summary for visual-feature drift, where both 1st- and 2nd-order shifts are common.

5.3 Maximum Mean Discrepancy with a Gaussian kernel

MMD is used in the same streaming 2-window framework as the Fréchet score: at time t , the previous and current embedding windows are compared, the resulting scalar MMD score is appended to the score history, and the adaptive quantile threshold determines the alarm. Therefore, this detector also monitors changes in the input vectors x_t rather than prediction errors.

The squared Maximum Mean Discrepancy [10] between 2 samples $X = \{x_1, \dots, x_n\}$ and $Y = \{y_1, \dots, y_m\}$ with respect to a positive-definite kernel $k(\cdot, \cdot)$ admits the unbiased empirical estimator

$$\widehat{\text{MMD}}_u^2(X, Y) = \frac{1}{n(n-1)} \sum_{i \neq j} k(x_i, x_j) + \frac{1}{m(m-1)} \sum_{i \neq j} k(y_i, y_j) - \frac{2}{nm} \sum_{i, j} k(x_i, y_j). \quad (11)$$

The implementation uses the Gaussian RBF kernel $k(x, y) = \exp(-\gamma \|x - y\|_2^2)$. The 2 within-group sums in Equation (11) are realised by setting the diagonal to 0 of the corresponding kernel matrix and dividing by $n(n-1)$ rather than n^2 , which yields the unbiased estimator without the diagonal self-similarities. The cross term is simply the mean of the cross kernel matrix.

The kernel bandwidth γ is set per call by the *median heuristic* [9]: pairwise squared distances are computed on the union of the 2 windows (sub-sampled to a fixed cap with a deterministic seed for stability), the strict upper triangle is extracted, non-positive entries are dropped, and $\gamma = 1/\text{median}$ of the surviving squared distances. This adapts the kernel to the local scale of the embedding distribution at every comparison, which matters because the 2-window framework feeds the empirical distribution of past scores back into the threshold: a slowly changing scale would otherwise inflate the threshold quantile and reduce sensitivity. Optionally, the bandwidth can be fixed in advance to disable the heuristic, although the default behaviour is data-driven.

The unbiased finite-sample estimate in Equation (11) can be slightly negative because the diagonal self-similarities are omitted. The implementation clips the reported score to 0 for plotting and thresholding. High values indicate that the kernel mean embeddings of the 2 windows are far apart in the reproducing-kernel Hilbert space induced by the Gaussian kernel [10], which captures differences in the underlying distributions and not only differences in the mean.

5.4 Kolmogorov–Smirnov windowing on a scalar projection

Projected KSWIN reduces each embedding to a scalar and feeds the resulting univariate stream into a sliding Kolmogorov–Smirnov windowing test [19]. 3 projections are available:

- the ℓ_2 norm of the embedding, sensitive only to magnitude.
- the mean of the embedding components, a generic linear projection.

- the ℓ_2 distance from a running exponentially weighted moving average of the embedding,

$$z_t = \|x_t - c_{t-1}\|_2, \quad c_t = (1 - \alpha) c_{t-1} + \alpha x_t,$$

initialised lazily with the initial incoming embedding. This default projection captures a *distance-from-recent-typical* signal: when the underlying class distribution shifts, the embedding stops agreeing with the running centroid and the projected scalar grows.

The univariate stream is then fed into the Kolmogorov–Smirnov windowing test. The detector keeps a sliding window over the projected scalar. At each new sample, the most recent sub-window is compared against a random sub-sample of the older portion of the sliding window using the 2-sample Kolmogorov–Smirnov statistic [15], and a drift is declared when the test rejects the null hypothesis of equal distributions at the configured significance level. The detector exposes a 1-sided score by reporting 1 minus the test p -value, so that the drift-score plot has a horizontal cut-off corresponding to the statistical decision boundary.

The Kolmogorov–Smirnov windowing test is used here as a drift-only test. It raises alarms but does not provide a separate warning state.

6 Online classifiers and replacement strategy

In the performance regime the detector input is no longer the multivariate embedding but the binary per-sample classification error of an online classifier. The detector’s task is to spot a change in the prequential error process [7].

6.1 Underlying online classifiers

The performance benchmark uses 3 streaming classifiers, all implemented through the `river` framework [17] and evaluated on the same prequential stream:

- **Logistic regression.** A running standardiser is followed by a 1-vs-rest online logistic regressor [20]. The standardiser updates a per-coordinate running mean and variance using Welford’s online algorithm [22]. The classifier is updated sample-by-sample with stochastic gradient descent on the log-loss [2]. The 1-vs-rest wrapper makes the same pipeline valid for both binary streams and the multi-class `GarbageDataset`.
- **Hoeffding Tree.** A running standardiser is followed by River’s incremental Hoeffding Tree classifier. This gives a classical tree-based streaming baseline whose splits are learned online from the same dense multimodal embedding vectors.
- **Gaussian Naive Bayes.** A running standardiser is followed by River’s Gaussian Naive Bayes classifier. This provides a simple probabilistic streaming baseline with online class-conditional Gaussian estimates.

The pipeline is scored *prequentially*: for every sample the monitor predicts (test) before it updates with the true label (train), following the standard prequential evaluation protocol for streaming classifiers [7]. This is the only training source: there is no held-out fit phase, so the reported classifier metrics are prequential metrics on the synthetic stream.

6.2 Dual-model replacement strategy

The performance monitor maintains 2 parallel pipelines on the same stream:

- **Never-reset model.** The online classifier learns from every sample regardless of detector flags. Its per-sample error is the natural detector input, because its trajectory is unaffected by the detector’s own decisions: the detector never reads back a signal that depends on its own past decisions, which would otherwise create a feedback loop. The model is never replaced, so this strategy represents the behaviour of a classical online learner that trusts the stream and relies on plasticity alone.
- **Replace-on-drift model with a warned shadow.** The same classifier class is run in parallel, but is reactive to the detector flags. On a *warning* flag a fresh shadow model is instantiated (if none exists) and starts learning from every subsequent sample in parallel with the active model. On a *drift* flag, the active model is discarded and the shadow is promoted in its place, then the shadow is cleared. This warm-shadow scheme follows the warning-then-drift design used by classical drift detectors such as DDM [6].

In the River implementations used here, this warm-shadow path is available only when the detector exposes a warning state. HDDM-W supports such warnings. ADWIN, KSWIN and Page-Hinkley are used as drift-only detectors in this benchmark. For those drift-only detectors the replacement model is necessarily *cold*: it is created only after the drift alarm and has not been trained during an earlier warning phase. All result tables mark these cold-replacement configurations with an asterisk (*).

For every sample, the pipeline records the prediction, error and running accuracy of each of the 2 strategies, the cumulative number of model replacements, and the per-sample wall-clock costs of prediction and update for the active model and the shadow separately. The cost of training the shadow is reported as part of the replace-on-drift strategy, because that cost is intrinsic to it: without a hot shadow, the swap on drift would promote a cold model.

The warning threshold is therefore not just a detector parameter. It also controls how much training data the shadow model receives before a possible promotion. A more sensitive warning setting can start the shadow earlier, so replacement may lead to better post-drift accuracy or macro- F_1 than the values reported here. The trade-off is that earlier warnings may also create more unnecessary shadow models and higher computational cost. The benchmark keeps the detector parameters fixed in order to compare methods under 1 transparent setting, but sensitivity tuning is a natural next step whenever Replace-on-drift is used as an adaptation strategy rather than only as a controlled diagnostic.

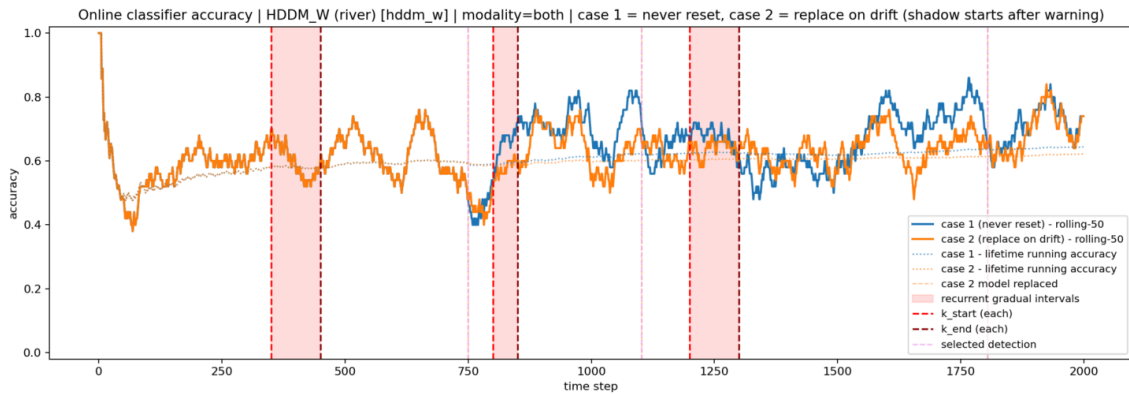


Figure 5: Illustrative prequential accuracy trace for the Never-reset and Replace-on-drift strategies. Solid lines show rolling-window accuracy and dotted lines show lifetime running accuracy. Shaded regions and dashed boundaries mark recurrent gradual drift intervals.

7 Performance-drift detectors

In the performance regime, the detector input collapses to a binary value per step. 4 detectors are available. Each is briefly described below, together with the parameter that governs the alarm rate.

7.1 Kolmogorov–Smirnov windowing

This is the same Kolmogorov–Smirnov windowing test as in Section 5.4 [19]. Only the input changes. The test compares a recent sub-window of the binary error signal to a random sub-sample of the older portion of the sliding window using the 2-sample Kolmogorov–Smirnov statistic [15], and fires drift when the test rejects the null at the configured significance level. The significance level is the main parameter governing the alarm rate: smaller values mean stricter rejection (fewer false alarms but more delay), larger values mean more sensitive detection (faster but noisier).

7.2 Adaptive Windowing

The Adaptive Windowing detector (ADWIN) [1] maintains a variable-length window over a univariate stream statistic and tests whether the older and newer parts of the window appear to come from the same distribution. In our experiments we used the implementation provided by `river.drift.ADWIN`. Thus, the detector was not implemented by explicitly evaluating a separate cut-threshold formula. At each step, the monitored statistic was passed to the detector via `update`, and a drift event was registered whenever River’s built-in flag `drift_detected` became true.

ADWIN adapts its window automatically: during stable periods the window can grow, while after a detected change the older part of the window is discarded so that subsequent estimates are based on more recent samples. The main sensitivity parameter is the significance value δ . Smaller values of δ make the detector more conservative, typically reducing false alarms at the cost of longer detection delay, whereas larger values make the detector more reactive but potentially noisier. Unless stated otherwise, the remaining implementation parameters were left at River’s defaults.

7.3 Page–Hinkley test

The Page–Hinkley test [18] is a CUSUM-style sequential test for changes in the mean of a stream. It maintains a running estimate of the mean, a cumulative sum of the centred residuals (with a small slack constant), and fires a drift alarm when the excursion of the cumulative statistic exceeds a fixed threshold. The main parameter is the threshold magnitude: the smaller the threshold, the more sensitive the detector. The detector does not emit warnings.

7.4 Hoeffding-bound drift detection (weighted)

The weighted Hoeffding-bound drift detector [5] is designed for binary error streams, which is exactly the signal produced by the never-reset prequential classifier. It maintains an exponentially weighted estimate of the error rate and uses Hoeffding’s inequality [14] to test whether the current weighted mean is significantly higher than the lowest weighted mean seen so far in the stable phase. 2 confidence levels control 2 thresholds: warnings fire when the looser bound is exceeded and drifts fire when the stricter bound is exceeded. The main parameter governing the alarm rate is the drift confidence (the alpha for the drift bound). The warning confidence is set larger so that the warning threshold is crossed before the drift threshold, which is the regime the warm-shadow strategy exploits to warm-start the replacement classifier (Section 6).

8 Interactive Streamlit UI

In addition to the batch benchmark scripts, the project includes an interactive Streamlit application for inspecting individual stream runs. From the repository root, the application can be opened with:

```
streamlit run multimodal_streamlit_app.py.
```

Streamlit prints a local browser URL after startup.

The interface is a workbench for configuring and visualising one streaming experiment at a time. The sidebar exposes the most important stream and detector parameters: drift target (**data** or **performance**), detector choice, modality, dataset, drift schedule, drift positions or gradual intervals, stream length, window size, detector thresholds, KSWIN projection settings, text-feature settings, image/text weights, dominant image-label probability, detection horizon, alarm-retention policy, random seed, device and output directory. It also shows the equivalent command-line invocation before running the experiment, so an interactive configuration can be reproduced later from the CLI as well.

After a run finishes, the app can reopen saved runs from the selected output directory. It displays the detected classes, selected detector, modality, drift target, predicted drift positions, recall, precision, F_1 , selected alarm count, detection delays, missed-detection rate and per-sample timing. For performance-target runs it also reports the online classifier strategies, including Never-reset and Replace-on-drift accuracy, replacement count, shadow activity and per-strategy timing. The visual panels show the stream scatter plot, the drift-score trace emitted by the selected detector, the per-sample processing-time plot and, for performance-target runs, the online classifier accuracy trace. The tabbed tables give a more detailed view of matched detections, false alarms, missed detections, the stream manifest, per-class counts, segment counts, sample stream items, raw drift-score rows, timing rows and the saved run configuration JSON.

The Streamlit application is therefore useful for checking how a generated stream actually looks before aggregating it into benchmark tables: where the true drift intervals occur, how class proportions change over time, when the detector emits warnings or drift alarms, whether alarms are repeated, and how the classifier replacement strategy behaves around those events. The final tables in Section 9 are still generated from the batch CSV outputs, but the UI provides the detailed run-level diagnostics used to interpret those aggregates.

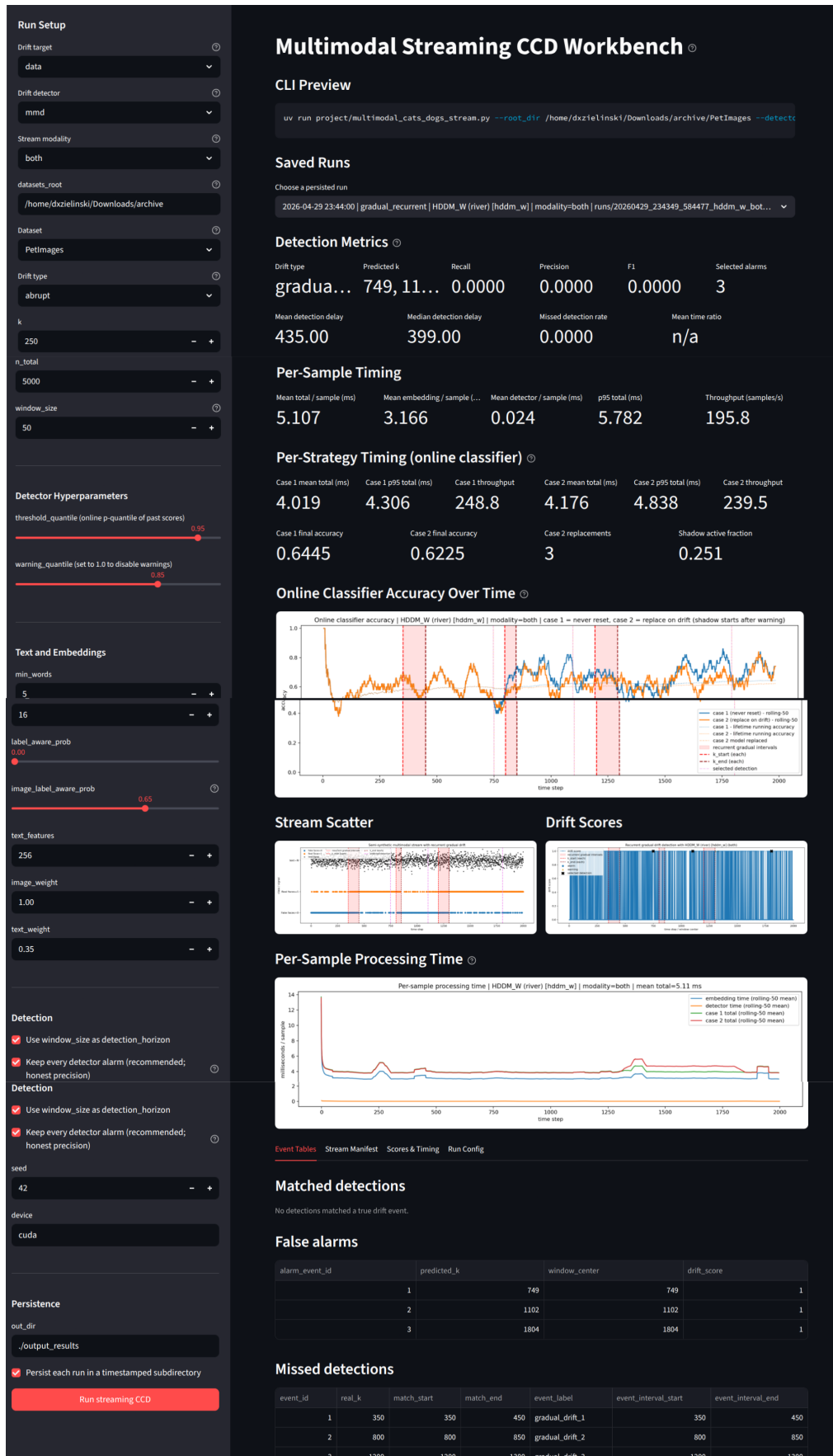


Figure 6: Screenshot of the Streamlit workbench showing the configurable stream parameters and run-level drift diagnostics.

9 Benchmarking results and source-expansion extension

This section reports the completed benchmark suite from `benchmark_output/final_comparison`. The final run contains 1800 granular trials: 3 datasets (`Faces`, `GarbageDataset` and `PetImages`), 4 dominant-class probabilities $p^* \in \{0.55, 0.65, 0.75, 0.85\}$, 3 data-drift detectors, 4 performance-drift detectors, 3 online classifiers for the performance regime, and 10 random-seed repetitions per compatible configuration. The `Faces` and `PetImages` streams are binary classification problems. `GarbageDataset` is multi-class, so it tests whether the same monitoring pipeline transfers beyond the binary setting. All runs use the multimodal stream with image and text features (`modality=both`) and the recurrent-gradual drift schedule. The final subsection reports an additional source-expansion experiment from `benchmark_output/final_comparison_extension`, designed to test a stronger form of drift in which entirely new datasets, and therefore new classes, enter the stream over time.

9.1 Experimental configuration

Each trial contains $N = 1000$ stream samples. The multimodal embedder concatenates ResNet-18 image features with a 256-dimensional hashed text representation and then ℓ_2 -normalises the resulting vector. The caption branch is included in every reported run because the benchmark uses `modality=both`. However, the experimental conclusions below are interpreted primarily as image-embedding drift results, because the captions are short synthetic tokens rather than human-written descriptions (the main goal of text features in this benchmark is to introduce some additional noise to make the detection and classification problems more difficult). For every dataset and p^* value, 10 random seeds are evaluated. Each within-dataset recurrent-gradual trial contains 3 true drift intervals. Their ramp locations are sampled anew per seed, so the reported standard deviations include both sampling variation and schedule variation.

In the **data-drift regime** the detector input is the ℓ_2 -normalised multimodal embedding itself. The 2-window detectors (MMD and Fréchet) use $W = 50$ for the window size, a drift threshold quantile of 0.95, a warning threshold quantile of 0.85, and the adaptive-quantile decision rule described in Section 5.1. MMD uses the Gaussian RBF kernel with bandwidth set by the median heuristic on the union of the 2 windows. KSWIN in the data regime is driven by the *centroid-distance projection* (the ℓ_2 distance between the incoming embedding and the EWMA running centroid, with smoothing factor $\alpha_{\text{ewma}} = 0.01$). The underlying `river.drift.KSWIN` uses $\alpha = 0.05$, `stat_size` = 30 and `window_size` = $\max(2 \cdot \text{stat_size}, W) = 100$. Detection is therefore based on changes in the input vectors x_t , not on classification error.

In the **performance-drift regime** the detector input is instead the binary per-sample 0/1 error signal of the online classifier. KSWIN reuses the same parameters listed above but consumes the binary error directly. ADWIN uses `river.drift.ADWIN` with $\delta = 0.02$, `clock` = 32, `max_buckets` = 5, `min_window_length` = 5 and `grace_period` = 10. Page-Hinkley uses `min_instances` = 30, $\delta = 0.005$, threshold $\lambda = 18$ and forgetting factor $\alpha = 0.9999$. The weighted Hoeffding-bound detector (HDDM-W) uses `drift_confidence` = 0.005, `warning_confidence` = 0.01, $\lambda = 0.05$, in 1-sided test mode.

The detection metrics and classifier metrics have different meanings. Detector recall is event-level: a true drift event is counted as detected if at least 1 alarm falls in its accepted interval. Detector precision is alarm-level: each alarm is judged against the true drift intervals. Detector F_1 combines those 2 quantities. By contrast, `classifier_precision`, `classifier_recall` and `classifier_f1` are macro-averaged classification metrics over the classes present in the stream. These aliases refer to the Never-reset classifier. The explicit `classifier_case1_*` (Never-reset) and `classifier_case2_*` (Replace-on-drift) columns report the 2 classifier strategies separately.

Within each dataset block, or within each sweep table, a marked value is the best value for that metric among the rows being compared. Higher values are better for detection and classification precision, recall and F_1 . Lower values are better for latency, replacement counts and runtime.

Alarm volume is reported in the detector tables as an alarm-drift gap, computed per trial as raw alarms minus the true number of drift events and then averaged. This column is scored by absolute distance from 0: smaller absolute gaps are better, and ties are counted for all tied rows. The Best column is a descriptive count of the scored per-metric wins, including ties. It should not be read as a formal significance test: several comparisons have large standard deviations, so the count is mainly a readable summary of the observed trade-offs. In performance-target tables, an asterisk (*) next to a detector or a Replace-on-drift column means that the corresponding replacement run used a cold classifier because the detector did not expose a warning state.

9.2 Data-drift detection

Table 1 and Figure 7 summarize the representative $p^* = 0.65$ setting. MMD is the strongest detector on **Faces** and **PetImages**, with mean F_1 values of 0.56 and 0.70, respectively. On **GarbageDataset**, Fréchet and MMD are close, with Fréchet slightly ahead in F_1 (0.60 versus 0.58) and precision (0.63 versus 0.52). KSWIN is consistently the cheapest detector and raises far fewer alarms, but this comes with lower recall and lower F_1 across all 3 datasets. The 2 multivariate 2-window detectors therefore remain preferable when detection quality is the primary criterion, whereas KSWIN is useful as a low-cost baseline.

Table 1: Data-target benchmark results for recurrent-gradual drift. Values are mean $\pm$ standard deviation over 10 trials, except per-sample time, which is shown as a mean. Marked values are the best within each dataset block. Higher is better for recall, precision and F_1 , while lower is better for delay and ms/sample. The alarm-drift gap is raw alarms per trial minus true drift events per trial, so positive values mean extra alarms and negative values mean fewer alarms than true events by count. For this gap, values closer to 0 are better. The Best column counts the marked wins, including ties.

Dataset	Detector	Recall	Precision	F_1	Delay	Alarms – drifts	ms/sample	Best
Faces	Fréchet	0.65 ± 0.21	0.38 ± 0.23	0.45 ± 0.19	87.6 ± 46.0	72.1 ± 22.5	4.89	0
Faces	MMD	0.77 ± 0.35	0.50 ± 0.25	0.56 ± 0.25	58.1 ± 66.5	53.3 ± 31.8	38.54	4
Faces	KSWIN	0.48 ± 0.36	0.27 ± 0.20	0.34 ± 0.25	136.0 ± 89.5	3.9 ± 1.5	3.42	2
GarbageDataset	Fréchet	0.62 ± 0.19	0.63 ± 0.24	0.60 ± 0.17	51.0 ± 19.1	85.2 ± 30.4	4.04	3
GarbageDataset	MMD	0.73 ± 0.31	0.52 ± 0.21	0.58 ± 0.25	79.0 ± 42.7	63.5 ± 27.8	37.89	1
GarbageDataset	KSWIN	0.55 ± 0.39	0.26 ± 0.18	0.35 ± 0.24	82.5 ± 42.8	4.0 ± 1.2	2.33	2
PetImages	Fréchet	0.70 ± 0.23	0.47 ± 0.31	0.52 ± 0.28	65.5 ± 27.8	90.1 ± 26.3	3.95	0
PetImages	MMD	0.80 ± 0.28	0.66 ± 0.20	0.70 ± 0.22	47.1 ± 19.4	59.4 ± 31.1	40.13	4
PetImages	KSWIN	0.62 ± 0.40	0.27 ± 0.18	0.37 ± 0.24	96.2 ± 62.1	4.2 ± 2.4	3.02	2

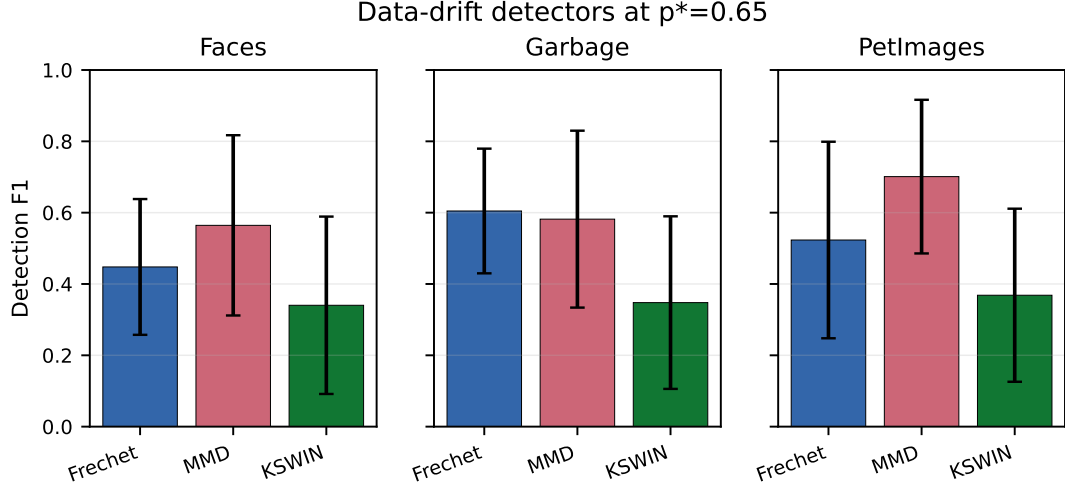


Figure 7: Data-drift detector F_1 at $p^* = 0.65$. Bars show means and error bars show 1 standard deviation over 10 random-seed repetitions for each dataset–detector pair. The plot visually confirms the table-level pattern: MMD is strongest on Faces and PetImages, Fréchet is marginally strongest on GarbageDataset, and KSWIN is consistently weaker in detection quality.

Figure 8 shows the full p^* sweep for the data-drift regime. The response is not perfectly monotonic because these are finite, soft, recurrent-gradual streams with randomly sampled ramp locations. Nevertheless, the sweep confirms that the detectors are not tuned to 1 drift magnitude. **PetImages** is weakest at $p^* = 0.55$ and improves sharply once the dominant class becomes more separable. **GarbageDataset** is more stable across the sweep, with average data-detector F_1 between 0.49 and 0.53. **Faces** shows higher variance and remains the most ambiguous binary dataset.

Table 2: Data-target F_1 across the full p^* sweep, averaged over the 3 data-drift detectors. Values are mean $\pm$ standard deviation over detector–trial combinations. The Best column counts the number of dominant-probability settings tied for the highest displayed mean within each dataset row.

Dataset	$p^* = 0.55$	$p^* = 0.65$	$p^* = 0.75$	$p^* = 0.85$	Best
Faces	0.35 ± 0.26	0.45 ± 0.24	0.43 ± 0.22	0.44 ± 0.24	1
GarbageDataset	0.53 ± 0.23	0.51 ± 0.25	0.49 ± 0.21	0.53 ± 0.23	2
PetImages	0.31 ± 0.21	0.53 ± 0.27	0.57 ± 0.21	0.54 ± 0.24	1

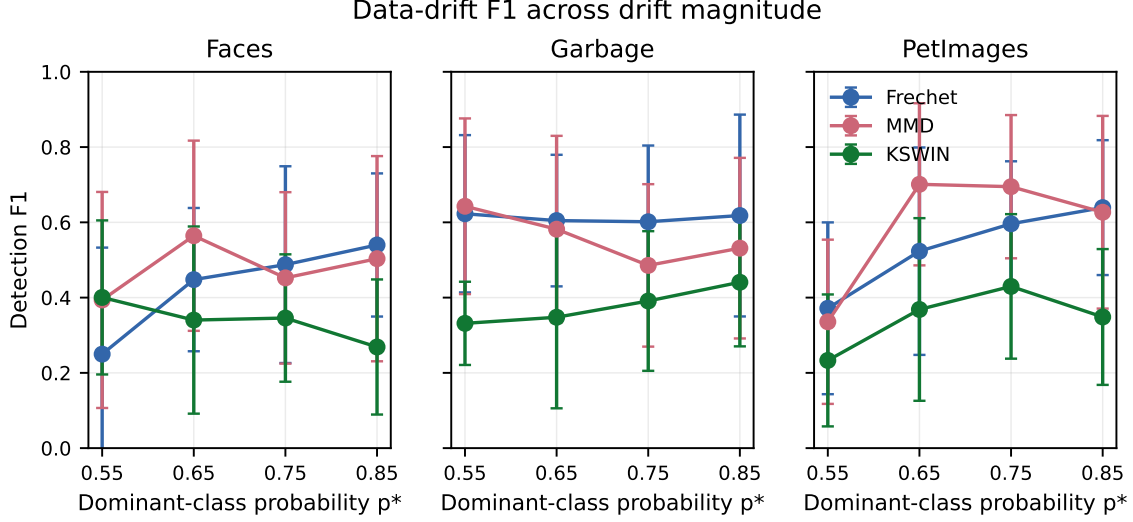


Figure 8: Data-drift detector F_1 over all tested dominant-class probabilities. Each panel corresponds to 1 dataset and each line to 1 detector. Points are trial means and error bars are 1 standard deviation over 10 trials. The sweep shows that drift magnitude interacts with dataset geometry rather than producing a uniformly monotonic response.

9.3 Performance-drift detection

The performance-drift regime is harder because the detectors see only the binary prequential error of the online classifier. Table 3 and Figure 9 show the representative $p^* = 0.65$ setting after averaging over the 3 online classifiers. On **Faces**, HDDM-W gives the strongest performance-drift detection ($F_1 = 0.43 \pm 0.30$). On **GarbageDataset**, Page-Hinkley is best ($F_1 = 0.34 \pm 0.26$), while ADWIN follows. On **PetImages**, the classification problem is so well solved that the error stream contains little drift signal. All performance detectors stay near 0. The asterisk beside ADWIN, KSWIN and Page-Hinkley indicates that their Replace-on-drift results use cold replacement rather than warning-trained shadow promotion.

Table 3: Performance-target benchmark results for recurrent-gradual drift. Values are mean $\pm$ standard deviation over classifier-trial combinations. Never-reset is the baseline classifier that is never replaced. Replace-on-drift promotes a warning-trained shadow only for detectors with warning support. Detector names marked with * use a cold classifier at replacement time. Marked values are best within each dataset block. Higher is better for recall, precision, detector F_1 and classifier F_1 . The alarm-drift gap is raw alarms minus true drift events for each classifier-trial row, so positive values mean extra alarms and negative values mean fewer alarms than true events by count. For this gap, values closer to 0 are better. The Best column counts the marked wins, including ties.

Dataset	Detector	Recall	Precision	F_1	Alarms – drifts	Never-reset F_1	Replace-on-drift F_1	Best
Faces	ADWIN*	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	-2.60 $\pm$ 0.50	0.612 $\pm$ 0.015	0.612 $\pm$ 0.015	2
Faces	HDDM-W	0.39 $\pm$ 0.30	0.52 $\pm$ 0.38	0.43 $\pm$ 0.30	-0.67 $\pm$ 0.80	0.612 $\pm$ 0.015	0.610 $\pm$ 0.016	5
Faces	KSWIN*	0.18 $\pm$ 0.29	0.23 $\pm$ 0.36	0.19 $\pm$ 0.29	-1.27 $\pm$ 1.53	0.612 $\pm$ 0.015	0.609 $\pm$ 0.015	1
Faces	Page-Hinkley*	0.18 $\pm$ 0.24	0.23 $\pm$ 0.31	0.19 $\pm$ 0.25	-1.23 $\pm$ 1.07	0.612 $\pm$ 0.015	0.607 $\pm$ 0.016	1
GarbageDataset	ADWIN*	0.22 $\pm$ 0.21	0.50 $\pm$ 0.47	0.29 $\pm$ 0.27	-1.63 $\pm$ 0.72	0.629 $\pm$ 0.082	0.532 $\pm$ 0.149	2
GarbageDataset	HDDM-W	0.12 $\pm$ 0.18	0.27 $\pm$ 0.41	0.16 $\pm$ 0.24	-2.27 $\pm$ 0.87	0.629 $\pm$ 0.082	0.581 $\pm$ 0.145	2
GarbageDataset	KSWIN*	0.08 $\pm$ 0.16	0.11 $\pm$ 0.24	0.08 $\pm$ 0.18	-2.13 $\pm$ 1.20	0.629 $\pm$ 0.082	0.566 $\pm$ 0.169	1
GarbageDataset	Page-Hinkley*	0.31 $\pm$ 0.23	0.43 $\pm$ 0.36	0.34 $\pm$ 0.26	-0.87 $\pm$ 0.94	0.629 $\pm$ 0.082	0.496 $\pm$ 0.171	4
PetImages	ADWIN*	0.07 $\pm$ 0.15	0.20 $\pm$ 0.41	0.11 $\pm$ 0.22	-2.27 $\pm$ 0.64	0.962 $\pm$ 0.012	0.952 $\pm$ 0.026	1
PetImages	HDDM-W	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	-2.60 $\pm$ 0.50	0.962 $\pm$ 0.012	0.962 $\pm$ 0.012	2
PetImages	KSWIN*	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	-2.53 $\pm$ 0.57	0.962 $\pm$ 0.012	0.960 $\pm$ 0.014	1
PetImages	Page-Hinkley*	0.10 $\pm$ 0.17	0.27 $\pm$ 0.45	0.14 $\pm$ 0.25	-1.87 $\pm$ 0.63	0.962 $\pm$ 0.012	0.949 $\pm$ 0.022	5

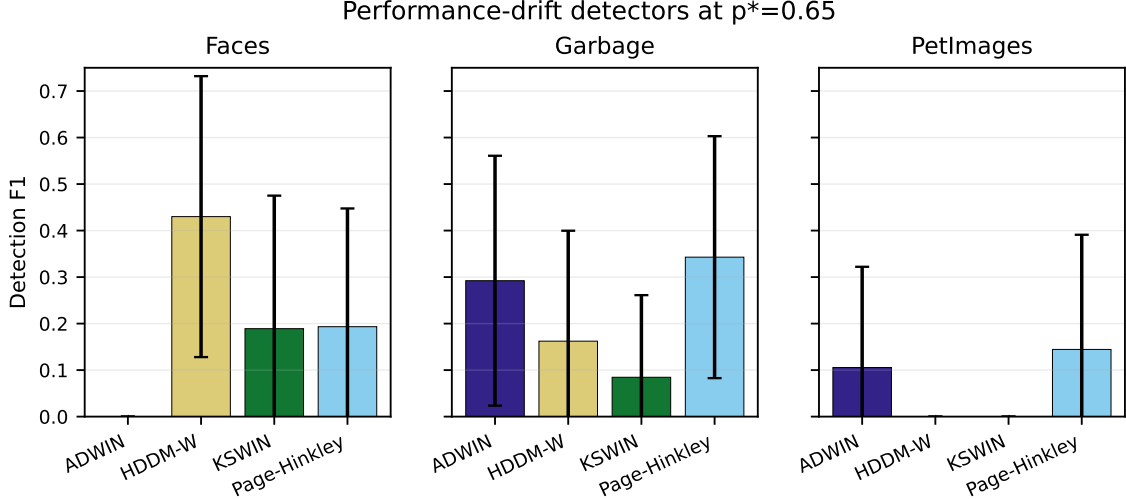


Figure 9: Performance-drift detector F_1 at $p^* = 0.65$, averaged over logistic regression, Hoeffding Tree and Gaussian Naive Bayes. Bars show means and error bars show 1 standard deviation over classifier-trial combinations. The lower values relative to Figure 7 show that the binary error stream is a weaker drift signal than the full embedding distribution.

The classifier columns in Table 3 should be interpreted as downstream context for the detector, not as additional detector scores. The Never-reset macro- F_1 is independent of the detector decision rule and therefore often ties across detector rows. The Replace-on-drift macro- F_1 does depend on the detector, because false, late or missing alarms determine when the active model is replaced. This is why detector F_1 and classifier F_1 can point in different directions: a detector can raise useful drift alarms while still causing too many unhelpful classifier resets.

Figure 10 shows that performance-drift detection remains classifier-signal-limited across the full p^* sweep. The mean performance-detector F_1 on **PetImages** stays below 0.09 for all tested drift magnitudes because the classifiers maintain very high macro- F_1 . **Faces** and **GarbageDataset** improve slightly as p^* increases, but the error bars remain large, indicating that the detector choice, classifier choice and exact ramp placement all matter.

Table 4: Performance-target F_1 across the full p^* sweep, averaged over all performance detectors and classifiers. Values are mean $\pm$ standard deviation over detector-classifier-trial combinations. The Best column counts the number of dominant-probability settings tied for the highest displayed mean within each dataset row.

Dataset	$p^* = 0.55$	$p^* = 0.65$	$p^* = 0.75$	$p^* = 0.85$	Best
Faces	0.17 ± 0.27	0.20 ± 0.29	0.29 ± 0.32	0.34 ± 0.31	1
GarbageDataset	0.21 ± 0.28	0.22 ± 0.26	0.24 ± 0.30	0.28 ± 0.33	1
PetImages	0.06 ± 0.18	0.06 ± 0.17	0.09 ± 0.21	0.05 ± 0.18	1

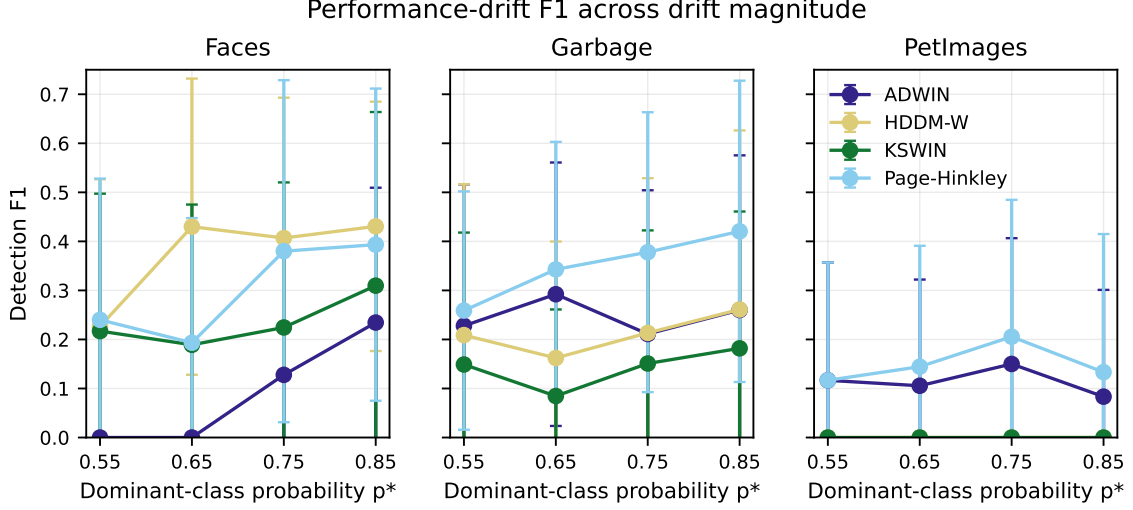


Figure 10: Performance-drift detector F_1 over all tested dominant-class probabilities. Points are means and error bars are 1 standard deviation over classifier-trial combinations. The large error bars show that performance detection depends not only on drift magnitude, but also on how strongly the chosen classifier’s error process reacts to that drift.

9.4 Classifier-side comparison

Table 5 and Figure 11 compare the Never-reset classifier with the Replace-on-drift classifier at $p^* = 0.65$. Logistic regression and Gaussian Naive Bayes are the strongest classifiers on **GarbageDataset**. Logistic regression is best on **PetImages**, and Gaussian Naive Bayes is slightly best on **Faces**. The replace-on-drift strategy does not systematically improve classifier performance. It is almost neutral on **Faces**, mildly negative on **PetImages**, and strongly negative for the **GarbageDataset** Hoeffding Tree, where mean macro- F_1 drops from 0.521 to 0.330. This is consistent with the detector results: false or late alarms can reset a model that was already adapting online, and a cold replacement is especially risky because it starts without the warning-period training that a shadow model would have received.

Table 5: Classifier strategy comparison at $p^* = 0.65$. Precision, recall and F_1 are macro-averaged over classes present in the stream. Values are mean $\pm$ standard deviation over detector-trial combinations. Replace-on-drift columns marked with * aggregate both warm-shadow runs (HDDM-W) and cold-replacement runs (ADWIN, KSWIN and Page-Hinkley). Marked values are best within each dataset block. Higher is better except for replacements, where lower is better. The Best column counts these per-metric wins, including ties.

Dataset	Classifier	Never-reset F_1	Replace-on-drift F_1^*	Never-reset P	Replace-on-drift P^*	Never-reset R	Replace-on-drift R^*	Replacements	Best
Faces	Logistic regression	0.617 ± 0.008	0.613 ± 0.010	0.618 ± 0.008	0.613 ± 0.010	0.618 ± 0.008	0.613 ± 0.010	0.80 ± 0.91	1
Faces	Hoeffding Tree	0.598 ± 0.013	0.598 ± 0.014	0.633 ± 0.012	0.616 ± 0.020	0.613 ± 0.010	0.606 ± 0.014	1.40 ± 1.39	1
Faces	Gaussian NB	0.621 ± 0.011	0.617 ± 0.016	0.632 ± 0.011	0.623 ± 0.017	0.626 ± 0.009	0.620 ± 0.015	1.27 ± 1.20	5
GarbageDataset	Logistic regression	0.669 ± 0.028	0.643 ± 0.039	0.671 ± 0.029	0.642 ± 0.042	0.673 ± 0.027	0.650 ± 0.036	0.62 ± 0.70	2
GarbageDataset	Hoeffding Tree	0.521 ± 0.024	0.330 ± 0.062	0.633 ± 0.019	0.527 ± 0.109	0.483 ± 0.028	0.326 ± 0.046	1.98 ± 0.83	0
GarbageDataset	Gaussian NB	0.696 ± 0.022	0.659 ± 0.050	0.723 ± 0.028	0.708 ± 0.039	0.682 ± 0.021	0.635 ± 0.057	0.62 ± 0.70	6
PetImages	Logistic regression	0.971 ± 0.006	0.970 ± 0.006	0.971 ± 0.006	0.970 ± 0.007	0.972 ± 0.006	0.971 ± 0.006	0.15 ± 0.36	7
PetImages	Hoeffding Tree	0.948 ± 0.006	0.933 ± 0.016	0.948 ± 0.007	0.933 ± 0.016	0.949 ± 0.005	0.934 ± 0.016	0.53 ± 0.51	0
PetImages	Gaussian NB	0.965 ± 0.007	0.963 ± 0.009	0.965 ± 0.008	0.964 ± 0.009	0.966 ± 0.006	0.964 ± 0.008	0.17 ± 0.38	0

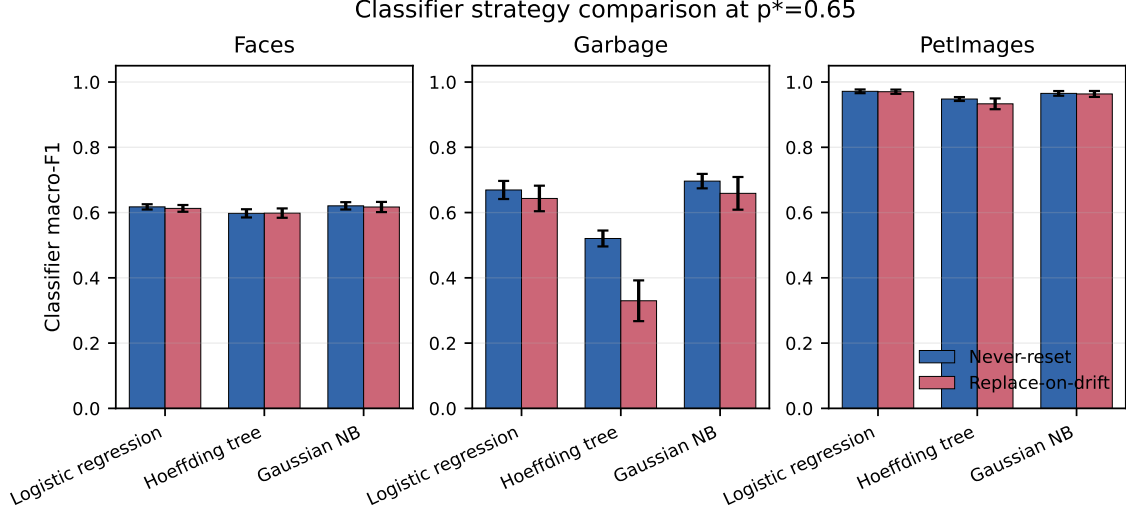


Figure 11: Classifier macro- F_1 at $p^* = 0.65$ for the Never-reset and Replace-on-drift strategies. Bars show means and error bars show 1 standard deviation over detector-trial combinations. The plot highlights that full replacement is not systematically beneficial: it is nearly neutral on Faces, harmful for the GarbageDataset Hoeffding Tree, and slightly harmful on PetImages.

Table 6 isolates the classifier-side comparison to the HDDM-W detector. This removes the cold-replacement configurations from Table 5: every Replace-on-drift column in the new table is computed from the detector path that can promote a warning-trained shadow model. The star on the Replace-on-drift headers is local to this table and marks this HDDM-W-only warm-shadow subset.

Table 6: Classifier strategy comparison at $p^* = 0.65$ using only HDDM-W performance-drift alarms. Precision, recall and F_1 are macro-averaged over classes present in the stream. Values are mean $\pm$ standard deviation over 10 trials. Replace-on-drift columns are the HDDM-W warm-shadow results. Marked values are best within each dataset block. Higher is better except for replacements, where lower is better.

Dataset	Classifier	Never-reset F_1	Replace-on-drift F_1	Never-reset P	Replace-on-drift P	Never-reset R	Replace-on-drift R	Replacements	Best
Faces	Logistic regression	0.617 $\pm$ 0.008	0.612 $\pm$ 0.014	0.618 $\pm$ 0.008	0.612 $\pm$ 0.014	0.618 $\pm$ 0.008	0.612 $\pm$ 0.014	1.40 $\pm$ 0.70	1
Faces	Hoeffding Tree	0.598 $\pm$ 0.013	0.601 $\pm$ 0.016	0.633 $\pm$ 0.013	0.606 $\pm$ 0.016	0.613 $\pm$ 0.010	0.603 $\pm$ 0.015	2.30 $\pm$ 0.67	1
Faces	Gaussian NB	0.621 $\pm$ 0.012	0.617 $\pm$ 0.016	0.632 $\pm$ 0.011	0.619 $\pm$ 0.016	0.626 $\pm$ 0.010	0.618 $\pm$ 0.016	2.10 $\pm$ 0.74	5
GarbageDataset	Logistic regression	0.669 $\pm$ 0.029	0.669 $\pm$ 0.029	0.671 $\pm$ 0.030	0.671 $\pm$ 0.030	0.673 $\pm$ 0.028	0.673 $\pm$ 0.028	0.00 $\pm$ 0.00	1
GarbageDataset	Hoeffding Tree	0.521 $\pm$ 0.025	0.385 $\pm$ 0.037	0.633 $\pm$ 0.020	0.580 $\pm$ 0.046	0.483 $\pm$ 0.030	0.366 $\pm$ 0.031	1.50 $\pm$ 0.53	0
GarbageDataset	Gaussian NB	0.696 $\pm$ 0.023	0.690 $\pm$ 0.034	0.723 $\pm$ 0.029	0.720 $\pm$ 0.034	0.682 $\pm$ 0.021	0.674 $\pm$ 0.036	0.10 $\pm$ 0.32	6
PetImages	Logistic regression	0.971 $\pm$ 0.006	0.971 $\pm$ 0.006	0.971 $\pm$ 0.006	0.971 $\pm$ 0.006	0.972 $\pm$ 0.006	0.972 $\pm$ 0.006	0.00 $\pm$ 0.00	7
PetImages	Hoeffding Tree	0.948 $\pm$ 0.006	0.948 $\pm$ 0.006	0.948 $\pm$ 0.007	0.948 $\pm$ 0.007	0.949 $\pm$ 0.005	0.949 $\pm$ 0.005	0.00 $\pm$ 0.00	1
PetImages	Gaussian NB	0.965 $\pm$ 0.007	0.965 $\pm$ 0.007	0.965 $\pm$ 0.008	0.965 $\pm$ 0.008	0.966 $\pm$ 0.006	0.966 $\pm$ 0.006	0.00 $\pm$ 0.00	1

The warm-shadow subset preserves the main conclusion, but clarifies the cause. On Faces, HDDM-W replacement is nearly neutral and even gives the Hoeffding Tree a small macro- F_1 increase, from 0.598 to 0.601. On GarbageDataset, however, the Hoeffding Tree still falls sharply from 0.521 to 0.385, while Gaussian Naive Bayes remains close to its Never-reset baseline. On PetImages, HDDM-W raises no replacement alarms in these runs, so the 2 strategies tie exactly. Thus the weak Replace-on-drift result is not explained only by cold starts: a warm shadow helps when it receives useful warning-period examples, but replacing a model that was already learning online can still discard valuable class knowledge.

9.5 Source-expansion drift extension

The preceding experiments deliberately used soft label-distribution drifts inside a fixed dataset. That is a useful controlled setting, but it also explains an important limitation of the classifier-side results: the Never-reset model had already seen all classes before and after each drift. Even when the dominant class changed, the minority classes were still present with probability $1 - p^*$, so the

online learner could accumulate experience on every class throughout the stream. In that regime, Replace-on-drift rarely has a natural advantage. A reset discards a model that may already know the minority classes and may already be adapting smoothly to the new class proportions. This is why the previous classifier-side comparison found Replace-on-drift comparable to, or slightly weaker than, Never-reset.

To stress this assumption, an additional benchmark introduces a stronger *source-expansion drift*. The stream starts with images from **Faces**. The initial gradual drift introduces a new source, **PetImages**. The following gradual drift introduces **GarbageDataset**. Labels are made dataset-specific, so the classifier does not solve a reused 2-class or 10-class task. Instead, the extended stream has 14 global labels: 2 **Faces** classes, 2 **PetImages** classes and 10 **GarbageDataset** classes. The dominant labels in the 3 stable stages are the local class-0 labels of the successive sources: **Faces**: class 'Fake', **PetImages**: class 'Cat' and **GarbageDataset**: class 'battery'.

Formally, let S_0, S_1, S_2 denote **Faces**, **PetImages** and **GarbageDataset**. Let C_j be the class set of source S_j . Let $d_j \in C_j$ be the local class-0 label of source S_j . In stable stage j , the active class set is $A_j = C_0 \cup \dots \cup C_j$. The stage distribution is

$$q_j(y) = \begin{cases} p^*, & y = d_j, \\ \frac{1-p^*}{|A_j|-1}, & y \in A_j \setminus \{d_j\}, \\ 0, & y \notin A_j. \end{cases}$$

During each gradual drift interval $[a_j, b_j]$, the sampling distribution is linearly interpolated from the old stage distribution to the new distribution,

$$q_t = (1 - \alpha_t)q_{j-1} + \alpha_t q_j, \quad \alpha_t = \frac{t - a_j}{b_j - a_j}.$$

Thus the new source does not appear abruptly in 1 sample, but its classes genuinely have 0 probability before their introduction. This is the key difference from the original benchmark: after the initial source change, the classifier must start learning labels it could not have observed previously.

The completed extension results use the same multimodal representation (**modality=both**), $N = 1000$, window size $W = 50$, the same detector parameters as Section 9.1, 4 p^* values $\{0.55, 0.65, 0.75, 0.85\}$ and 10 random seeds. Each trial contains 2 gradual source-expansion events, with the ramp intervals sampled anew per seed. The output contains 600 granular runs: for every p^* value and trial, 3 data-target detectors are evaluated, and the 4 performance-target detectors are evaluated with 3 online classifiers.

Table 7 shows the data-target detector results at $p^* = 0.65$. The stronger source-level shift is much easier for embedding-level monitoring than the within-dataset label-proportion drift. Fréchet detects every true source-expansion event (Recall = 1.00) and reaches $F_1 = 0.76 \pm 0.18$. This high sensitivity comes with many repeated alarms, as indicated by the large positive alarm-drift gap. Therefore Fréchet is best interpreted here as a sensitive detector rather than as a sparse alarm policy. MMD follows ($F_1 = 0.68 \pm 0.27$), while KSWIN is again the low-alarm, low-cost option (6.6 ± 1.2 more alarms than true drift events and 2.87 ms/sample) but has the weakest F_1 . Compared with the original recurrent-gradual benchmark at $p^* = 0.65$, the average data-target F_1 increases from about 0.50 to 0.63, which confirms that changing the source distribution is a more visible data drift than changing class proportions inside 1 fixed dataset.

Table 7: Source-expansion data-target results at $p^* = 0.65$. Values are mean $\pm$ standard deviation over 10 trials. The alarm-drift gap is raw alarms minus the 2 true gradual source-expansion events. Values closer to 0 are better. The Best column counts marked metric wins, including ties at the displayed precision.

Detector	Recall	Precision	F_1	Delay	Alarms – drifts	ms/sample	Best
Fréchet	1.00 $\pm$ 0.00	0.64 $\pm$ 0.21	0.76 $\pm$ 0.18	29.5 $\pm$ 15.2	196.3 $\pm$ 44.9	5.42	4
MMD	0.75 $\pm$ 0.26	0.64 $\pm$ 0.27	0.68 $\pm$ 0.27	48.9 $\pm$ 27.6	69.2 $\pm$ 11.7	39.43	1
KSWIN	0.95 $\pm$ 0.16	0.31 $\pm$ 0.07	0.47 $\pm$ 0.09	54.0 $\pm$ 26.1	6.6 $\pm$ 1.2	2.87	2

The performance-target results in Table 8 show that the stronger data shift also becomes more visible in the classifier error stream. At $p^* = 0.65$, average performance-detector F_1 rises from about 0.16 in the original benchmark to about 0.40 in the source-expansion benchmark. HDDM-W is the clearest winner, with $F_1 = 0.62 \pm 0.40$, the highest recall and the highest precision. The large standard deviations are informative rather than merely noise: they indicate substantial dependence on the underlying classifier, because the performance detector observes the Never-reset classifier’s error process, not the embedding distribution directly. If a classifier reacts sharply to new unseen classes, the drift becomes easy to detect. If the classifier adapts smoothly or remains uncertain across several classes, the error signal is weaker.

Table 8: Source-expansion performance-target results at $p^* = 0.65$, averaged over logistic regression, Hoeffding Tree and Gaussian Naive Bayes. Values are mean $\pm$ standard deviation over classifier-trial combinations. Detector names marked with * use cold replacement because no warning state is available. The alarm-drift gap is scored by closeness to 0, and the Best column counts marked wins, including ties.

Detector	Recall	Precision	F_1	Alarms – drifts	Never-reset F_1	Replace-on-drift F_1	Best
ADWIN*	0.23 $\pm$ 0.39	0.16 $\pm$ 0.26	0.19 $\pm$ 0.31	−0.17 $\pm$ 1.15	0.410 $\pm$ 0.143	0.383 $\pm$ 0.164	2
HDDM-W	0.57 $\pm$ 0.41	0.72 $\pm$ 0.45	0.62 $\pm$ 0.40	−0.70 $\pm$ 0.92	0.410 $\pm$ 0.143	0.388 $\pm$ 0.152	5
KSWIN*	0.45 $\pm$ 0.40	0.41 $\pm$ 0.37	0.40 $\pm$ 0.34	−0.17 $\pm$ 1.82	0.410 $\pm$ 0.143	0.387 $\pm$ 0.165	2
Page-Hinkley*	0.48 $\pm$ 0.38	0.37 $\pm$ 0.31	0.39 $\pm$ 0.29	0.73 $\pm$ 1.36	0.410 $\pm$ 0.143	0.352 $\pm$ 0.152	1

The most important classifier-side result is more subtle. The stronger source-expansion drift does make alarms more meaningful, but it still does not make naive Replace-on-drift superior in macro- F_1 . Table 9 compares the 2 classifier strategies at $p^* = 0.65$. For all 3 classifiers, Replace-on-drift remains below Never-reset in macro- F_1 . The gap is small for logistic regression (−0.017), larger for Gaussian Naive Bayes (−0.029), and largest for the Hoeffding Tree (−0.051). However, the Hoeffding Tree also shows a higher ordinary accuracy after replacement (0.602 versus 0.545). This is a useful diagnostic: replacement can help the model track the dominant current source, but it can hurt macro- F_1 because the source-expanded stream still contains old classes as minority classes after each drift. A reset throws away knowledge about those older classes, and macro- F_1 penalises that loss even when accuracy on the dominant source improves.

Table 9: Source-expansion classifier-side comparison at $p^* = 0.65$, averaged over performance detectors and 10 trials. ΔF_1 is Replace-on-drift minus Never-reset. Replace-on-drift columns marked with * aggregate both warm-shadow and cold-replacement detector configurations. The Best column counts marked wins. Higher is better except for replacements, where lower is better.

Classifier	Never-reset F_1	Replace-on-drift F_1 *	ΔF_1	Never-reset Acc.	Replace-on-drift Acc. *	Replacements	Best
Logistic regression	0.550 $\pm$ 0.017	0.533 $\pm$ 0.039	−0.017 $\pm$ 0.035	0.769	0.753	0.97 $\pm$ 0.77	6
Hoeffding Tree	0.222 $\pm$ 0.027	0.172 $\pm$ 0.016	−0.051 $\pm$ 0.024	0.545	0.602	3.40 $\pm$ 1.24	0
Gaussian NB	0.458 $\pm$ 0.040	0.428 $\pm$ 0.051	−0.029 $\pm$ 0.044	0.746	0.738	1.40 $\pm$ 0.84	0

Table 10 repeats the source-expansion classifier comparison with only HDDM-W. This is the warm-shadow performance-detector setting for the extension, so it separates the effect of warning-trained promotion from the cold replacements used by ADWIN, KSWIN and Page-Hinkley in the mixed detector average.

Table 10: Source-expansion classifier strategy comparison at $p^* = 0.65$ using only HDDM-W performance-drift alarms. Values are mean $\pm$ standard deviation over 10 trials. Replace-on-drift columns are the HDDM-W warm-shadow results. Marked values are best within the source-expansion block. Higher is better except for replacements, where lower is better.

Classifier	Never-reset F_1	Replace-on-drift F_1	Never-reset P	Replace-on-drift P	Never-reset R	Replace-on-drift R	Replacements	Best
Logistic regression	0.550 $\pm$ 0.017	0.536 $\pm$ 0.034	0.576 $\pm$ 0.018	0.554 $\pm$ 0.046	0.537 $\pm$ 0.019	0.530 $\pm$ 0.030	0.50 $\pm$ 0.53	5
Hoeffding Tree	0.222 $\pm$ 0.028	0.188 $\pm$ 0.017	0.282 $\pm$ 0.041	0.208 $\pm$ 0.019	0.223 $\pm$ 0.030	0.200 $\pm$ 0.016	2.20 $\pm$ 0.63	0
Gaussian NB	0.458 $\pm$ 0.041	0.441 $\pm$ 0.036	0.614 $\pm$ 0.075	0.585 $\pm$ 0.053	0.415 $\pm$ 0.034	0.403 $\pm$ 0.029	1.20 $\pm$ 0.63	2

The HDDM-W-only view shows a modest improvement over the mixed cold/warm average, but not a reversal of the result. Logistic regression remains the strongest classifier and drops only from 0.550 to 0.536 in macro- F_1 . Gaussian Naive Bayes has the highest precision before and after replacement, but lower recall keeps its macro- F_1 below logistic regression. The Hoeffding Tree benefits from removing the cold detectors relative to Table 9, where its Replace-on-drift macro- F_1 was 0.172, but the HDDM-W warm-shadow value is still only 0.188 against a Never-reset value of 0.222. This reinforces the extension’s interpretation: warning-trained promotion reduces cold-start damage, yet full replacement still loses information about older sources that remain present as minority classes.

Finally, Table 11 summarises the extension over the full p^* sweep. The data-target detector average peaks at $p^* = 0.75$ ($F_1 = 0.71 \pm 0.17$), while the performance-target detector average is highest at the softest setting $p^* = 0.55$ ($F_1 = 0.42 \pm 0.34$) and then declines slightly as the dominant source becomes stronger. The classifier macro- F_1 also decreases as p^* increases because the stream becomes more imbalanced: the dominant class is easier to predict, but the many minority classes, especially after **GarbageDataset** enters, have fewer effective training examples. The Replace-on-drift penalty shrinks from -0.040 at $p^* = 0.55$ to nearly 0 at $p^* = 0.85$. At high p^* there is less minority-class performance to preserve, so losing old knowledge is less visible in macro- F_1 .

Table 11: Source-expansion sweep over p^* . Data detector F_1 is averaged over data-target detectors and trials. Performance detector F_1 is averaged over performance-target detectors, classifiers and trials. Replace-on-drift F_1 marked with * aggregates both warm-shadow and cold-replacement configurations. The Best column counts marked wins across the 5 reported metrics.

p^*	Data detector F_1	Performance detector F_1	Never-reset F_1	Replace-on-drift F_1^*	ΔF_1	Best
0.55	0.59 $\pm$ 0.21	0.42 $\pm$ 0.34	0.446	0.406	-0.040 ± 0.046	3
0.65	0.63 $\pm$ 0.23	0.40 $\pm$ 0.37	0.410	0.378	-0.032 ± 0.037	0
0.75	0.71 $\pm$ 0.17	0.36 $\pm$ 0.38	0.343	0.334	-0.009 ± 0.029	1
0.85	0.66 $\pm$ 0.19	0.35 $\pm$ 0.39	0.292	0.290	-0.002 ± 0.033	1

The extension therefore supports the original suspicion but refines it. The original benchmark was indeed a mild form of drift for the classifier: every class remained visible somewhere in the stream, so Never-reset could learn continuously. Source-expansion drift is much more severe and makes both embedding-level and performance-level drift detection easier. Nevertheless, detected drift does not automatically imply that full classifier replacement is the best adaptation rule. If old classes remain part of the post-drift distribution, even as minorities, an online model with accumulated knowledge can still be preferable to a cold replacement. A stronger adaptation strategy would likely need to combine drift alarms with partial reset, replay, class-balanced updates or warm-start transfer rather than simply discarding the active model.

10 Conclusion

This report described an end-to-end streaming concept-drift benchmark on a multimodal classification stream: image-folder ingestion with deep visual features and hashed bag-of-words text features combined into an ℓ_2 -normalised multimodal vector, 4 within-dataset drift schedules

(abrupt, gradual, recurrent, recurrent gradual), an additional source-expansion drift in which new datasets and new classes enter the stream, 2 complementary monitoring regimes, and a dual prequential classifier [7] that feeds either KSWIN [19], Adaptive Windowing [1], Page–Hinkley [18], or weighted Hoeffding-bound detection [5]. The data-only regime uses 2 adaptive-quantile sliding-window detectors based on Maximum Mean Discrepancy [10] and the Fréchet distance [4], plus a projected KSWIN test [19]. The 2 classifier-side strategies are run in parallel: the Never-reset strategy is never reset, and the Replace-on-drift strategy is replaced on drift, using a warning-trained shadow when the detector supports warnings and a cold classifier otherwise. They are reported with separate accuracy and timing figures, so that the trade-off between plasticity-only and reset-on-drift learning can be measured on the same stream.

The completed experiments support 5 main conclusions. (1) Monitoring the multimodal embedding distribution is more reliable than monitoring classifier error alone for the within-dataset label-distribution drifts. MMD and Fréchet distance consistently outperform KSWIN in detection quality, while KSWIN remains attractive only when very low detector cost and low alarm counts are more important than recall. (2) The data-drift approach transfers from binary streams to the multi-class **GarbageDataset**: the best data-detector F_1 remains around 0.60 at $p^* = 0.65$, comparable to or better than the binary datasets. (3) The p^* sweep shows that drift magnitude interacts with dataset geometry and recurrent-gradual ramp placement. Stronger drifts are not always monotonically easier in a 10-trial finite benchmark, but the lowest-magnitude setting is clearly hardest for **PetImages**.

(4) The source-expansion extension confirms that the original within-dataset drift was relatively mild for the classifier. When the stream changes from **Faces** to **PetImages** and then to **GarbageDataset**, the incoming distribution contains classes that were literally absent before the drift. This stronger shift is easier to detect: at $p^* = 0.65$, average data-target F_1 increases from about 0.50 in the original benchmark to about 0.63 in the extension, and average performance-target F_1 increases from about 0.16 to about 0.40. (5) Even this stronger drift does not make naive Replace-on-drift reliably better in macro- F_1 . Full replacement can improve ordinary accuracy for a classifier such as the Hoeffding Tree, because it tracks the currently dominant source more aggressively, but it still discards knowledge about older classes that remain present as minority classes after the drift. Macro- F_1 exposes that loss.

Overall, the final benchmark broadens the study from an initial binary demonstration into a 3-dataset, 3-classifier, 4-magnitude comparison plus a source-expansion stress test. The strongest practical recommendation is to use embedding-level monitoring as the primary drift signal, report classifier-side metrics separately from detector metrics, and treat reset-on-drift adaptation as a design choice that must be validated empirically. A detected drift is not itself proof that the active classifier should be discarded. In streams where old concepts remain relevant, better adaptation may require partial resets, replay, warm-start transfer or class-balanced updates rather than a replacement.

A Feedback discussion and mapping of revisions

This section explicitly connects the review remarks to the corresponding changes in the report. I also include 2 final clean-up decisions that are important for interpreting the revised experiments: the earlier fallback feature-extraction idea is not part of the final method, and the earlier “selected detection” plotting idea was removed from the main evaluation. The source-expansion drift experiment in Section 9.5 was added because it directly checks whether the original within-dataset label-prior drift was too mild for classifier replacement to be useful.

Review remark 1. Pseudocode and stream construction.

The report now contains pseudocode for the sample-by-sample stream builder and for the causal streaming loop. The stream-construction text explicitly separates the sorted class-to-ID mapping from the stochastic sample order: class IDs are assigned once from sorted folder names, but

each stream sample is drawn from the schedule’s categorical distribution. Thus, examples from different classes are interleaved. They are not placed as all of class A, then all of class B. Changed sections: Sections 2 and 3.

Review remark 2. Sorted class identifiers versus apparently random early samples.

This is now clarified as a difference between *label encoding* and *stream order*. The numeric identifier is deterministic, while the early samples look mixed because the sampler already draws from a soft class probability distribution. Changed section: Section 2.

Review remark 3. Origin of class-aware text tokens.

The text-stream subsection now states that class-aware tokens are manually curated short word lists keyed by class name. They are not extracted from the image content and are not generated by an image-captioning model. Changed section: Section 2.

Review remark 4. Earlier note about the deep-learning environment.

The technical report no longer describes a fallback feature extractor as part of the method, because that would distract from the final experiments. In an early prototype I tested a simpler representation while resolving environment problems with the built-in DriftLens multimodal model. That prototype was an environmental workaround, not a final experimental condition. Changed section: this feedback section only.

Review remark 5. Detector inputs, detector parameters and drift probabilities.

The report now distinguishes data-target detection, where the detector observes the embedding vector x_t , from performance-target detection, where it observes the binary prequential error of the Never-reset classifier. The experimental configuration lists the detector parameters used in the final runs. The drift-generation text also replaces the extreme pure switch with a soft dominant-class probability and reports the sweep $p^* \in \{0.55, 0.65, 0.75, 0.85\}$. Changed sections: Sections 3, 4.2 and 9.1.

Review remark 6. Meaning and frequency of predicted detections.

The main text now defines a predicted detection as a raw drift alarm, i.e. a stream time step for which the detector returns `is_drift=1`. The metrics use the complete alarm stream. Frequent alarms are expected for sensitive 2-window detectors on gradual drift because adjacent windows can remain different for many consecutive steps inside and after a ramp. Changed sections: Sections 1, 4.2 and 9.5.

Review remark 7. Fréchet distance on a stream of instances.

The Fréchet detector is now described as a 2-window empirical distribution detector. At each step, adjacent embedding windows are fitted with empirical means and covariances, and the sample-space Fréchet/Bures score is thresholded by the adaptive quantile rule. Changed sections: Sections 5.1 and 5.2.

Review remark 8. Details for MMD and KSWIN.

The MMD subsection now states that MMD is computed on adjacent embedding windows and thresholded by the same adaptive score quantile as the Fréchet detector. The KSWIN subsection describes the scalar projection, the EWMA centroid-distance signal and the univariate KS

windowing test. The experimental configuration gives the concrete KSWIN parameters. Changed sections: Sections 5.3, 5.4 and 9.1.

Review remark 9. Classifier baselines beyond logistic regression.

The classifier-side benchmark now includes logistic regression, Hoeffding Tree and Gaussian Naive Bayes, all evaluated prequentially through the same River-based interface. This makes the performance-target results less tied to 1 classifier family. Changed sections: Sections 6 and 9.4.

Review remark 10. Earlier “selected detection” idea.

I removed this terminology from the main report. It came from an earlier visualisation idea in which only the earliest K alarms would be displayed or kept. That is not well matched to a real stream, because K would be an arbitrary number chosen after deciding how many detections should be shown. The final evaluation is therefore fairer and simpler: all detector alarms are kept, and the metrics decide which alarms are useful. Changed section: this feedback section only.

Review remark 11. Experiment beyond binary classification.

The final benchmark includes the multi-class `GarbageDataset`. The source-expansion extension goes further by creating a 14-label stream in which new datasets and new classes enter over time. Changed sections: Sections 9 and 9.5.

Review remark 12. Plot/table descriptions and result interpretation.

The results section now explains what each table and plot measures, how error bars should be read, which values are best, and why a detector may perform well while the Replace-on-drift classifier still loses macro- F_1 . The conclusion follows the same cause–result chain: stream shift changes the embedding distribution. Sometimes it also changes the classifier error process, and classifier replacement is useful only when the adaptation rule preserves enough relevant prior knowledge. Changed sections: Sections 9 and 10.

Additional final-report refinement 13. Cold replacement and detector-warning support.

The performance-target tables now mark drift-only detector configurations with an asterisk (*). In these rows, Replace-on-drift cannot promote a warning-trained shadow model, because the detector does not expose warning states in the used River interface. Instead, replacement starts from a cold classifier after the drift alarm. The text also notes that making a warning threshold more sensitive could start shadow training earlier and can improve post-drift classifier accuracy or macro- F_1 , although at the cost of more warnings and extra computation. Changed sections: Sections 6, 9.3, 9.4 and 9.5.

Additional final-report refinement 14. Interactive Streamlit user interface.

A new section now documents the interactive Streamlit application that was developed alongside the batch benchmark scripts. The application is a single-run workbench: it exposes the stream and detector parameters in a sidebar, runs one streaming experiment at a time, and visualises the sampled class labels, the per-step drift score, the decision threshold and the raised alarms, so that a single configuration can be inspected interactively rather than only through the aggregate result tables. This makes the pipeline easier to explore and the per-step detector behaviour easier to interpret. Changed section: Section 8.

Additional final-report refinement 15. New source-expansion drift type.

A new drift mechanism, *source-expansion drift*, was developed and added to the benchmark. In contrast to the within-dataset label-prior schedules of Section 3, which only re-weight a fixed set of classes, a source-expansion event is the moment at which entirely new classes from a new dataset are introduced into the current stream. The older classes are still present after the event, but they now appear as minority classes, while the newly introduced classes carry probability mass that was exactly 0 before the event. This produces a stronger, more realistic shift in which the classifier must start learning labels it could not have observed previously, and it is the regime in which classifier replacement finally may have a natural advantage. Changed section: Section 9.5.

References

- [1] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the SIAM International Conference on Data Mining*, 2007.
- [2] Léon Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT 2010*, pages 177–186. Physica-Verlag HD, 2010.
- [3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [4] D. C. Dowson and B. V. Landau. The fréchet distance between multivariate normal distributions. *Journal of Multivariate Analysis*, 12(3):450–455, 1982.
- [5] Isvani Frías-Blanco, José del Campo-Ávila, Gonzalo Ramos-Jiménez, Rafael Morales-Bueno, Agustín Ortiz-Díaz, and Yailé Caballero-Mota. Online and non-parametric drift detection methods based on hoeffding’s bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823, 2015.
- [6] João Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. Learning with drift detection. In *Advances in Artificial Intelligence – SBIA 2004*, pages 286–295. Springer, 2004.
- [7] João Gama, Raquel Sebastião, and Pedro Pereira Rodrigues. On evaluating stream learning algorithms. *Machine Learning*, 90(3):317–346, 2013.
- [8] João Gama, Indre Zliobaite, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):44:1–44:37, 2014.
- [9] Damien Garreau, Wittawat Jitkrittum, and Motonobu Kanagawa. Large sample analysis of the median heuristic. arXiv preprint arXiv:1707.07269, 2017.
- [10] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13:723–773, 2012.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [12] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in Neural Information Processing Systems*, 2017.
- [13] Nicholas J. Higham. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, 2008.
- [14] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.
- [15] Frank J. Massey. The kolmogorov-smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253):68–78, 1951.
- [16] Leandro L. Minku and Xin Yao. Ddd: A new ensemble approach for dealing with concept drift. *IEEE Transactions on Knowledge and Data Engineering*, 24(4):619–633, 2012.
- [17] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, and Albert Bifet. River: Machine learning for streaming data in python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.

- [18] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.
- [19] Christoph Raab, Moritz Heusinger, and Frank-Michael Schleif. Reactive soft prototype computing for concept drift streams. In *Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*, 2020.
- [20] Ryan Rifkin and Aldebaro Klautau. In defense of one-vs-all classification. *Journal of Machine Learning Research*, 5:101–141, 2004.
- [21] Kilian Q. Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the International Conference on Machine Learning*, pages 1113–1120, 2009.
- [22] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.

List of Figures

1	Abrupt drift diagnostic. The red dashed line is the true change point. Magenta dashed lines are detector alarms. Because $p^* = 0.65$, both classes remain interleaved before and after the change, but the dominant class flips at $k = 100$.	4
2	Gradual drift diagnostic. The shaded interval is the true transition window, with red dashed boundaries at k_{start} and k_{end} . Magenta dashed lines are detector alarms. The class mixture changes continuously rather than at a boundary.	5
3	Recurrent abrupt drift diagnostic. Red dashed lines mark successive true change points and the dominant class alternates after each boundary. Magenta dashed lines show the detector alarms raised on this illustrative run.	5
4	Recurrent gradual drift diagnostic. Each shaded region is a true gradual transition interval, the red dashed lines mark ramp starts and ends, and magenta dashed lines show the detector alarms. This schedule combines recurrent concepts with delayed, ramp-like transitions.	6
5	Illustrative prequential accuracy trace for the Never-reset and Replace-on-drift strategies. Solid lines show rolling-window accuracy and dotted lines show lifetime running accuracy. Shaded regions and dashed boundaries mark recurrent gradual drift intervals.	12
6	Screenshot of the Streamlit workbench showing the configurable stream parameters and run-level drift diagnostics.	15
7	Data-drift detector F_1 at $p^* = 0.65$. Bars show means and error bars show 1 standard deviation over 10 random-seed repetitions for each dataset–detector pair. The plot visually confirms the table-level pattern: MMD is strongest on Faces and PetImages, Fréchet is marginally strongest on GarbageDataset, and KSWIN is consistently weaker in detection quality.	18
8	Data-drift detector F_1 over all tested dominant-class probabilities. Each panel corresponds to 1 dataset and each line to 1 detector. Points are trial means and error bars are 1 standard deviation over 10 trials. The sweep shows that drift magnitude interacts with dataset geometry rather than producing a uniformly monotonic response.	19
9	Performance-drift detector F_1 at $p^* = 0.65$, averaged over logistic regression, Hoeffding Tree and Gaussian Naive Bayes. Bars show means and error bars show 1 standard deviation over classifier–trial combinations. The lower values relative to Figure 7 show that the binary error stream is a weaker drift signal than the full embedding distribution.	20
10	Performance-drift detector F_1 over all tested dominant-class probabilities. Points are means and error bars are 1 standard deviation over classifier–trial combinations. The large error bars show that performance detection depends not only on drift magnitude, but also on how strongly the chosen classifier’s error process reacts to that drift.	21
11	Classifier macro- F_1 at $p^* = 0.65$ for the Never-reset and Replace-on-drift strategies. Bars show means and error bars show 1 standard deviation over detector–trial combinations. The plot highlights that full replacement is not systematically beneficial: it is nearly neutral on Faces, harmful for the GarbageDataset Hoeffding Tree, and slightly harmful on PetImages.	22

List of Tables

1	Data-target benchmark results for recurrent-gradual drift. Values are mean $\pm$ standard deviation over 10 trials, except per-sample time, which is shown as a mean. Marked values are the best within each dataset block. Higher is better for recall, precision and F_1 , while lower is better for delay and ms/sample. The alarm-drift gap is raw alarms per trial minus true drift events per trial, so positive values mean extra alarms and negative values mean fewer alarms than true events by count. For this gap, values closer to 0 are better. The Best column counts the marked wins, including ties.	17
2	Data-target F_1 across the full p^* sweep, averaged over the 3 data-drift detectors. Values are mean $\pm$ standard deviation over detector-trial combinations. The Best column counts the number of dominant-probability settings tied for the highest displayed mean within each dataset row.	18
3	Performance-target benchmark results for recurrent-gradual drift. Values are mean $\pm$ standard deviation over classifier-trial combinations. Never-reset is the baseline classifier that is never replaced. Replace-on-drift promotes a warning-trained shadow only for detectors with warning support. Detector names marked with * use a cold classifier at replacement time. Marked values are best within each dataset block. Higher is better for recall, precision, detector F_1 and classifier F_1 . The alarm-drift gap is raw alarms minus true drift events for each classifier-trial row, so positive values mean extra alarms and negative values mean fewer alarms than true events by count. For this gap, values closer to 0 are better. The Best column counts the marked wins, including ties.	19
4	Performance-target F_1 across the full p^* sweep, averaged over all performance detectors and classifiers. Values are mean $\pm$ standard deviation over detector-classifier-trial combinations. The Best column counts the number of dominant-probability settings tied for the highest displayed mean within each dataset row.	20
5	Classifier strategy comparison at $p^* = 0.65$. Precision, recall and F_1 are macro-averaged over classes present in the stream. Values are mean $\pm$ standard deviation over detector-trial combinations. Replace-on-drift columns marked with * aggregate both warm-shadow runs (HDDM-W) and cold-replacement runs (ADWIN, KSWIN and Page-Hinkley). Marked values are best within each dataset block. Higher is better except for replacements, where lower is better. The Best column counts these per-metric wins, including ties.	21
6	Classifier strategy comparison at $p^* = 0.65$ using only HDDM-W performance-drift alarms. Precision, recall and F_1 are macro-averaged over classes present in the stream. Values are mean $\pm$ standard deviation over 10 trials. Replace-on-drift columns are the HDDM-W warm-shadow results. Marked values are best within each dataset block. Higher is better except for replacements, where lower is better.	22
7	Source-expansion data-target results at $p^* = 0.65$. Values are mean $\pm$ standard deviation over 10 trials. The alarm-drift gap is raw alarms minus the 2 true gradual source-expansion events. Values closer to 0 are better. The Best column counts marked metric wins, including ties at the displayed precision.	24
8	Source-expansion performance-target results at $p^* = 0.65$, averaged over logistic regression, Hoeffding Tree and Gaussian Naive Bayes. Values are mean $\pm$ standard deviation over classifier-trial combinations. Detector names marked with * use cold replacement because no warning state is available. The alarm-drift gap is scored by closeness to 0, and the Best column counts marked wins, including ties.	24

9	Source-expansion classifier-side comparison at $p^* = 0.65$, averaged over performance detectors and 10 trials. ΔF_1 is Replace-on-drift minus Never-reset. Replace-on-drift columns marked with * aggregate both warm-shadow and cold-replacement detector configurations. The Best column counts marked wins. Higher is better except for replacements, where lower is better.	24
10	Source-expansion classifier strategy comparison at $p^* = 0.65$ using only HDDM-W performance-drift alarms. Values are mean $\pm$ standard deviation over 10 trials. Replace-on-drift columns are the HDDM-W warm-shadow results. Marked values are best within the source-expansion block. Higher is better except for replacements, where lower is better.	25
11	Source-expansion sweep over p^* . Data detector F_1 is averaged over data-target detectors and trials. Performance detector F_1 is averaged over performance-target detectors, classifiers and trials. Replace-on-drift F_1 marked with * aggregates both warm-shadow and cold-replacement configurations. The Best column counts marked wins across the 5 reported metrics.	25